MIT Libraries | DSpace@MIT

MIT Open Access Articles

The Random Ray Method for neutral particle transport

The MIT Faculty has made this article openly available. ***Please share*** how this access benefits you. Your story matters.

Citation:

Published Version: https://doi.org/10.1016/J.JCP.2017.04.038

Publisher: Elsevier BV

Permanent Link: https://hdl.handle.net/1721.1/135720

Version: Author's final manuscript

Terms of use: http://creativecommons.org/licenses/by-nc-nd/4.0/

Massachusetts Institute of Technology

The Random Ray Method for neutral particle transport

John R. Tramm[a,b,*], Kord S. Smith[a], Benoit Forget[a], Andrew R. Siegel[b]

[a] *Massachusetts Institute of Technology, Department of Nuclear Science & Engineering*
77 Massachusetts Avenue, 24-107, Cambridge, MA 02139
[b] *Argonne National Laboratory, Mathematics and Computer Science Department*
9700 S Cass Ave, Argonne, IL 60439

Abstract

A new approach to solving partial differential equations (PDEs) based on the Method of Characteristics (MOC) is presented. The Random Ray Method (TRRM) uses a stochastic rather than deterministic discretization of characteristic tracks to integrate the phase space of a problem. TRRM is potentially applicable in a number of transport simulation fields where long characteristic methods are used, such as neutron transport and gamma ray transport in reactor physics as well as radiative transfer in astrophysics. In this study, TRRM is developed and then tested on a series of exemplar reactor physics benchmark problems. The results show extreme improvements in memory efficiency compared to deterministic MOC methods, while also reducing algorithmic complexity, allowing for a sparser computational grid to be used while maintaining accuracy.

Keywords:
Method of Characteristics, Neutron Transport, Reactor Simulation, High Performance Computing, Stochastic Methods

1. Introduction

The Method of Characteristics (MOC) solves a partial differential equation (PDE) by defining characteristic lines (or curves) along which the PDE is reduced to an ordinary differential equation (ODE). By solving the ODE

*Corresponding author. Tel.: +1 (847) 421-1534.
Email addresses: jtramm@mit.edu (John R. Tramm), kord@mit.edu (Kord S. Smith), bforget@mit.edu (Benoit Forget), siegela@mcs.anl.gov (Andrew R. Siegel)

along a set of discrete characteristic lines (sometimes in conjunction with ray tracing) and iterating on the initial conditions for the lines, a solution to the governing PDE can be numerically estimated. MOC methods are popular in transport applications for problems featuring fine-grained spatial and angular dependencies. Such problems often feature steep gradients, voids, or optically thin regions of low diffusivity where more computationally efficient methods (such as diffusion and short characteristic methods) are subject to significant losses in accuracy or can become subject to ray effects. In these instances, MOC methods featuring long characteristics retain their fidelity as they can accurately capture global effects caused by fine-grained spatial and angular dependencies of the governing PDE.

Typical MOC applications will use a deterministic quadrature composed of many tracks to cover the phase space of the system evenly. The overall fidelity of the simulation is therefore related to the resolution of this quadrature. In this paper, a new stochastic method of track selection is presented wherein no deterministic quadrature is used. Rather, characteristic lines sampled from a uniform random distribution in phase space are followed through the system until termination criteria are met. The new method, known as The Random Ray Method (TRRM), can allow for increases in accuracy due to accumulation of effective resolution over successive iterations, while also reducing computational storage requirements by removing the need to store deterministic quadrature data and starting condition data.

The new method may be applicable to a range of fields of computational physics that utilize MOC techniques, such as neutron transport[1, 2, 3, 4], gamma ray transport[5, 6, 7], and astrophysics radiative transfer[8, 9, 10, 11]. To provide a preliminary assessment of the general utility of TRRM, the new method will be deployed in the context of a neutron transport application and tested on a set of reactor physics benchmark problems. TRRM will then be compared against a traditional deterministic neutron transport MOC application in order to quantify the benefits of TRRM.

1.1. Example Transport Problem Description

Neutron transport for the purposes of nuclear reactor simulation is an ideal test case for TRRM as there are numerous community benchmark problems available to test the accuracy and performance of new methods. Additionally, there are existing deterministic MOC based applications for neutron transport available to compare the performance of the new method against. Another reason to test the new method in this form is the opportunity for immediate

impact on the field of reactor physics, as the potential improvements offered by TRRM may enable high fidelity simulation of a full nuclear reactor core.

Full core simulations can potentially reduce design and construction costs, increase reactor performance and safety, reduce the amount of nuclear waste generated, and allow for much more complex and novel designs. To date, however, the time to solution and memory requirements for a full core high fidelity deterministic 3D calculation have rendered such calculations impractical, even using leadership class supercomputers. However, with High Performance Computing (HPC) architectures evolving rapidly towards exascale, the computational horsepower required to accomplish full 3D reactor simulations may soon be available.

MOC based simulation techniques are currently a field of active research in reactor physics as they have the potential for fast and efficient performance on a variety of next generation HPC systems, including CPU, GPU, and Intel Xeon Phi architectures[12, 13]. The task–based and highly vectorizable nature of MOC style simulations allows them to very efficiently utilize the floating point resources of modern hardware architectures – in one test reaching 63% of peak FLOPS available on a CPU node[12]. This is advantageous, as most high performance computing applications are only able to achieve 8 to 20% of peak FLOPS[14], with some neutronics algorithms, such as continuous energy Monte Carlo, often failing to exceed 1%[15]. While 2D MOC has long been used in reactor design and engineering as an efficient simulation method for smaller problems, the transition to 3D has only begun recently, and to our knowledge no 3D MOC based codes are currently used in industry. The delay of the onset of full 3D codes can be attributed to the impossibility of "naively" scaling current 2D codes into 3D due to prohibitively high memory requirements, though there is ongoing work into more optimal 3D MOC methods[1, 16, 17, 18, 19, 20].

Most current 2D MOC solvers take as input a constructive solid geometry (CSG) format to define the geometry specifications of the reactor. A pre-processing step is then performed which calculates an azimuthal quadrature over the 2D geometry, and converts the CSG definition into a much larger set of tracks and segment specifications known as a "tracking file." Numerical solution to the neutron transport equation can then be solved by sweeping angular fluxes along all tracks in the azimuthal quadrature, attenuating them and tallying their contributions to source regions along the way (as described in subsection 2.2). This method works well for small or medium sized problems in 2D, but the tracking file can be quite large. For example,

for a single 2D reactor pin cell with a fine quadrature the tracking file can be over 100MB. In such 2D methods, the 3rd dimension is assumed infinite and a polar quadrature is used when integrating the characteristic on each 2D segment. This 2D assumption works well for many traditional Pressurized Water Reactor (PWR) designs, but often cannot be applied to many next-generation reactor designs due to a much higher degree of axial heterogeneity or no symmetry basis being present at all (e.g., Pebble Bed reactors). Additionally, the angular flux at the boundaries of each track (i.e., where each track intersects the edge of the computational domain) must be stored in memory between each power iteration. The storage requirements for this can be even more than the requirements for storing the tracking file, especially in high fidelity calculations utilizing many neutron energy groups.

Naive extension of traditional 2D MOC methods into 3D is possible, but will result in unmanageably large tracking files and a large amount of track boundary data. For a full reactor core, this could be in excess of 1 PB of data just to store the tracking and segment information for a fine quadrature. Several advanced methods have been proposed to efficiently extend MOC into 3D. One method, developed by Gunow et al.[13], allows for a more memory-efficient extension into the 3rd dimension by using a 2D tracking file, but ray tracing on the fly in the axial direction using an extruded 2D geometry approach. This method inherently assumes a significant degree of axial homogeneity is present in order to maintain speed, which is often an acceptable assumption for traditional light water reactors. In addition, the performance penalty for extruded geometry is still unclear, and this methodology will not work for geometries lacking high degrees of axial symmetry. Such methods also do not reduce the memory requirements for storing the boundary angular flux data for all tracks. Alternative methods have been proposed, such as 2D MOC + 1D diffusion[21], or grouping similar length segments together to reduce storage overhead[22], though such methods often result in a loss of fidelity or a considerable increase in code complexity. No 3D MOC based methods that offer full 3D geometrical flexibility without considerable overhead have been proposed to date.

In this study, we present a new method of performing characteristic based numerical estimation of PDEs. The new method, known as The Random Ray Method (TRRM), defines ODEs that can be solved along characteristic lines through the phase space of the system (similar to MOC), but uses a stochastic rather than deterministic process for selecting which characteristic lines to solve. This new method has the potential to solve many of the challenges

associated with deterministic MOC neutron transport that have made full core 3D simulations difficult or prohibitively expensive.

To present the new method in the context of reactor simulation we will first provide background on the equations governing neutron transport as well as their standard discretizations when using the traditional deterministic form of the Method of Characteristics. Then, TRRM will be introduced along with its accompanying mathematical formalisms and pseudo-code. As TRRM is a stochastic method, convergence methodology and criteria are then addressed. In order to determine the viability of the new algorithm, the performance, accuracy, convergence, and time-to-solution characteristics of the algorithm as compared to traditional MOC methods are tested in a reference multi-threaded implementation. Finally, results from these benchmark simulations will be presented and discussed.

2. Background

As TRRM will be implemented in an example neutron transport application, this section provides an introduction to the equation governing neutron transport as well as its traditional deterministic MOC method of solution.

2.1. Reactor Simulation via the Neutron Transport Equation

Reactor simulation aims to calculate specific properties of the core both to make revisions to the initial reactor design and to better predict its operational behavior. Two of the most important phenomena are: 1) the eigenvalue, or criticality, of the reactor, and 2) its spatial power distribution. The eigenvalue, referred to as "k-effective", determines the ratio of neutron populations between successive generations within the reactor. K-effective therefore determines the balance between neutron production and loss. The power distribution within the reactor governs the thermal-hydraulic design considerations as well as the rate of burn-up of the nuclear fuel. The eigenvalue and power distribution are computed by numerically estimating the solution of the Boltzmann Neutron Transport Equation, which represents the balance of neutrons throughout the reactor in terms of their spatial location, the angular direction of their movement, and their energy level (or speed). In the case of a steady state reaction, we can write the Boltzmann Neutron Transport Equation as in Equation 1.

$$\vec{\Omega} \cdot \vec{\nabla}\psi(\vec{r}, \vec{\Omega}, E) + \Sigma_t(\vec{r}, E)\psi(\vec{r}, \vec{\Omega}, E) = \\ + \int_0^\infty dE' \int_{4\pi} d\Omega' \Sigma_s(\vec{r}, \vec{\Omega}' \to \vec{\Omega}, E' \to E)\psi(\vec{r}, \vec{\Omega}', E') \\ + \frac{\chi(\vec{r}, E)}{4\pi k_{eff}} \int_0^\infty dE' \nu\Sigma_f(\vec{r}, E') \int_{4\pi} d\Omega' \psi(\vec{r}, \vec{\Omega}', E') \quad (1)$$

Equation 1 defines the neutron transport equation where ψ is the angular neutron flux. This parameter represents the total distance travelled by all neutrons in a particular direction inside of a control volume, and is often given in units of $1/cm^2$. $\vec{\Omega}$ is the angular direction unit vector that represents the direction of travel for the neutron. $\vec{r}$ is the spatial position vector representing the location within the simulation. E is the neutron energy (or speed) in continuous space, which is often given in electron volts. Σ_t is the total neutron cross section. This value represents the total probability of interaction between a neutron travelling at a certain speed (i.e., neutron energy E) and a target nucleus (i.e., the material through which the neutron is travelling) per unit path length, typically given in units of $1/cm$. Cross section data is a combination of empirical data and quantum mechanical modelling employed in order to generate an evaluation represented either in pointwise form or resonance parameters for each target isotope of interest and is provided as input to a simulation. Σ_s is the scattering neutron cross section, which is similar to the total cross section but only measures scattering interactions between the neutron and the target nucleus, and includes extra dependencies on the change in angle and energy the neutron experiences as a result of the interaction. Σ_f is the fission neutron cross section, which is also similar to the total cross section but only measures the fission interaction between a neutron and a target nucleus. χ is the energy spectrum for neutrons born from fission, which represents a known distribution of outgoing neutron energies based on the material that fissioned, which is taken as input data to a computation. ν is the average number of neutrons born per fission. k_{eff} is the eigenvalue of the equation, representing the effective neutron multiplication factor. If the right hand side of Equation 1 is condensed into a single term, the represented by the total neutron source term $Q(\vec{r}, \vec{\Omega}, E)$, the form given in Equation 2 is

reached.

$$\overbrace{\vec{\Omega}\cdot\vec{\nabla}\psi(\vec{r},\vec{\Omega},E)}^{\text{streaming term}}+\overbrace{\Sigma_t(\vec{r},E)\psi(\vec{r},\vec{\Omega},E)}^{\text{absorption term}}=\overbrace{Q(\vec{r},\vec{\Omega},E)}^{\text{total neutron source term}} \tag{2}$$

2.2. The Method of Characteristics

In the context of reactor physics, the Method of Characteristics (MOC) approach is a numerical estimator for the Boltzman Neutron Transport Equation. It is an existing method to discretize some of the continuous dependencies in Equation 2 and to iteratively solve for the spatial distribution of neutrons throughout the system as well as the largest eigenvalue of the system. In this section, we present a brief introduction of the MOC technique, though a more robust description is given in [1, 2, 22, 23]. MOC works by solving Equation 2 along a single characteristic line, thus altering the full spatial and angular scope of the transport equation into something that holds true only for a particular linear path (or track) through the reactor. These tracks are linear for neutrons as they are neutral particles and are therefore not subject to field effects. To accomplish this, we parametrize $\vec{r}$ with respect to some reference location $\vec{r}_0$ such that $\vec{r}=\vec{r}_0+s\vec{\Omega}$. In this manner, Equation 2 can be rewritten for a specific segment length s at a specific angle $\vec{\Omega}$ through a constant cross section region of the reactor geometry as in Equation 3.

$$\vec{\Omega}\cdot\vec{\nabla}\psi(\vec{r}_0+s\vec{\Omega},\vec{\Omega},E)+\Sigma_t(\vec{r}_0+s\vec{\Omega},E)\psi(\vec{r}_0+s\vec{\Omega},\vec{\Omega},E)= Q(\vec{r}_0+s\vec{\Omega},\vec{\Omega},E) \tag{3}$$

As this equation holds along a 1 dimensional path, we can assume the dependence of s on $\vec{r}_0$ and $\vec{\Omega}$ such that $\vec{r}_0+s\vec{\Omega}$ simplifies to s. When the differential operator is also applied to the angular flux ψ, we arrive at the characteristic form of the Boltzmann Neutron Transport Equation given in Equation 4.

$$\frac{d}{ds}\psi(s,\vec{\Omega},E)+\Sigma_t(s,E)\psi(s,\vec{\Omega},E)=Q(s,\vec{\Omega},E) \tag{4}$$

An analytical solution to this characteristic equation can be achieved with the use of an integrating factor:

$$e^{-\int_0^s ds'\Sigma^T(s',E)}$$

to arrive at the final form of the characteristic equation shown in Equation 5.

$$\psi(s,\vec{\Omega},E) = \psi(\vec{r}_0,\vec{\Omega},E)e^{-\int_0^s ds'\Sigma_t(s',E)} + \int_0^s ds'' Q(s'',\vec{\Omega},E)e^{-\int_{s''}^s ds'\Sigma_t(s',E)} \tag{5}$$

With this characteristic form of the transport equation, we now have an analytical solution along a linear path through any constant cross section region of a reactor, with only the continuous energy dependence remaining as an issue to be addressed. Similar to many other solution approaches to the Boltzmann neutron transport equation, the MOC approach also uses a "multi-group" approximation in order to discretize the continuous energy spectrum of neutrons travelling through the reactor into fixed set of energy groups G, where each group $g \in G$ has its own specific cross section parameters. This makes the difficult non-linear continuous energy dependence much more manageable as group wise cross section data can be pre-computed and fed into a simulation as input data. Following the multi-group assumption, another assumption made is that a large and complex problem can be broken up into small constant cross section regions, and that these regions have group dependent isotropic sources (fission + scattering), Q_g. Anisotropic sources are also possible with MOC based methods[3], but are not used in this analysis for simplicity. With these key assumptions, the multi-group MOC form of the neutron transport equation can be written as in Equation 6.

$$\psi_g(s,\vec{\Omega}) = \psi_g(\vec{r_0},\vec{\Omega})e^{-\int_0^s ds'\Sigma_{tg}(s')} + \int_0^s ds'' Q_g(s'',\vec{\Omega})e^{-\int_{s''}^s ds'\Sigma_{tg}(s')} \tag{6}$$

The constructive solid geometry (CSG) definition of the reactor is used to create spatially defined source regions. These neutron source regions are often approximated as being constant (flat) in intensity of source, but can also be defined using a higher order source (linear, quadratic, etc.) that allows for fewer source regions to be required to achieve a specified solution fidelity[3, 24]. In this analysis, the normal approximation of a spatially constant isotropic fission and scattering source q_0 leads to simple exponential attenuation along an individual characteristic of length s given by Equation 7.

$$\psi_g(s) = \psi_g(0)e^{-\Sigma_{t,g}s} + \frac{q_0}{\Sigma_{t,g}}\left(1 - e^{-\Sigma_{t,g}s}\right) \tag{7}$$

These equations are applicable to single tracks through the reactor, though for an accurate simulation many tracks must be laid down to cover the continuous spatial and angular dependencies of the flux throughout the reactor. The power iteration method is used to sweep tracks through the domain, updating the source distribution after each iteration and thus iteratively converging towards the final source distribution throughout the reactor q_0. In order to discretize these spatial and angular dependencies by way of selecting specific tracks to sweep through, traditional 2D MOC codes use a fixed azimuthal quadrature that is laid down to cover the entire geometry with several different angles. Great care is taken to ensure that the 2D azimuthal quadrature contains cycles, and therefore that angular fluxes can continue indefinitely through the reactor between power iterations, only having to guess the angular flux once at the very beginning of the calculation. Tracks are then subdivided into "segments," where each segment corresponds to the path of a track through a single source region, as shown in Figure 1. An example of this conversion from CSG to tracking file is displayed in Figure 2. In traditional 2D MOC codes, this tracking and segmentation data is precomputed and stored in memory.

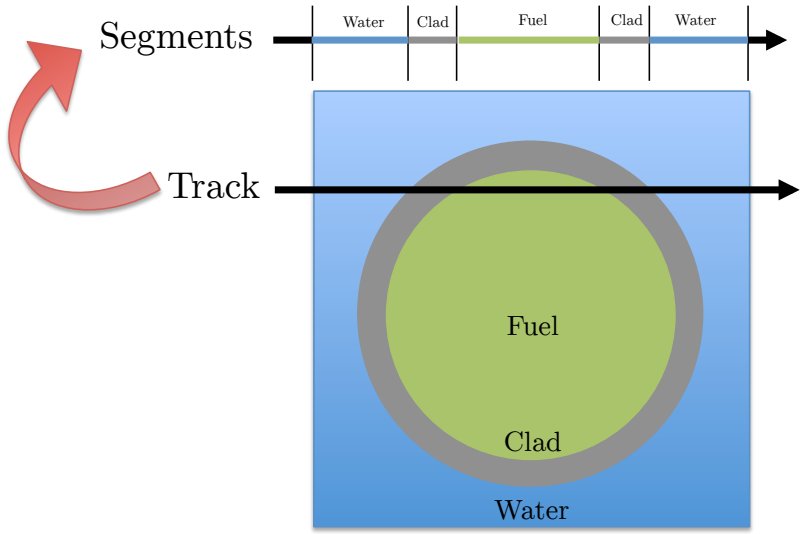
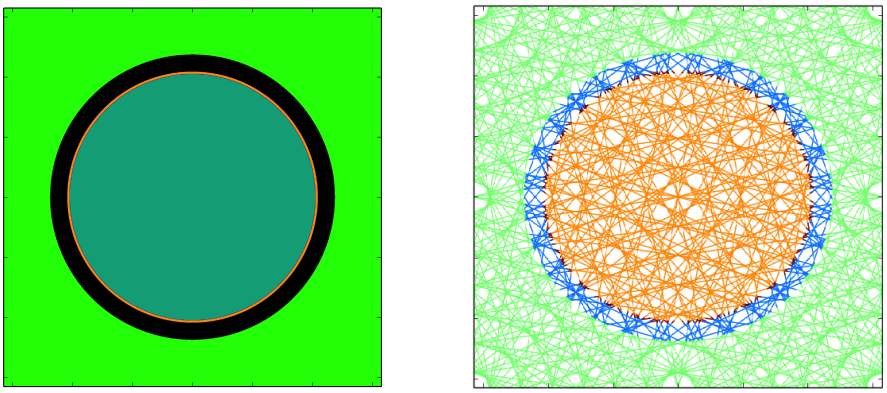


Figure 1: A single MOC track through a simplified reactor pin cell 2D geometry. This example shows the relationship between a constructive solid geometry, a track, and the segments that compose the track. An "intersection" is a further subdivision of a segment representing a single energy group of the flux.



(a) CSG pin cell **(b)** Cyclical quadrature tracking file

Figure 2: Example of a 2D Constructive Solid Geometry (CSG) definition of a reactor pin cell as resolved using a deterministic cyclical MOC quadrature. Each segment is explicitly stored in memory by traditional deterministic MOC codes.

High fidelity simulations require a high degree of accuracy from the quadrature selection. Accuracy is generally related to the track (or ray) density (i.e., how many segments are present in each flat source region being simulated). There are two factors that determine the ray density: the number of angles to integrate and the track spacing (i.e., the distance of separation between parallel tracks integrating the same angle). The angular flux in nuclear reactor lattice geometry is highly discontinuous, as shown in Figure 3, making it difficult to accurately integrate the overall angular flux using only a small set of discrete angles. As a result a large number of angles are desired to maximize accuracy. However, as angles are added, quadrature holes are formed due to the cyclical track requirement, which leads to uncovered areas in the geometry (and therefore volume error). These quadrature holes can be reduced with a smaller track spacing, but for an equal time-to-solution, the number of angles must also be reduced. Additionally, track spacing may also need to be reduced in order to accurately resolve the volume of very small geometrical features in the reactor design. Typically quadratures of 0.05 cm track spacing and 128 angles tend to be needed to achieve sufficient accuracy for traditional pressurized water reactor (PWR) pin cell designs featuring a fuel, gap, and clad style geometry.

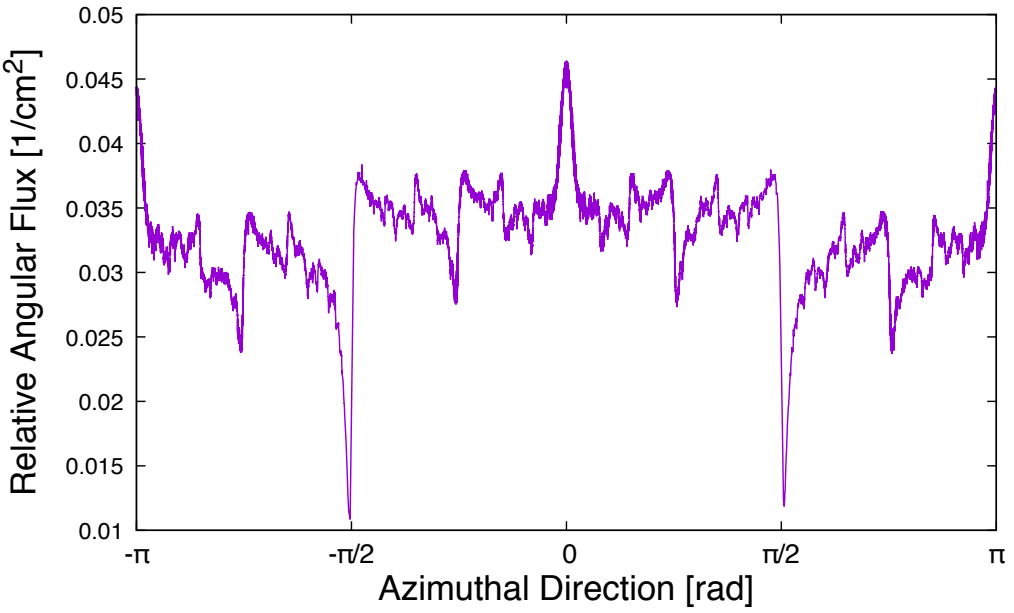


Figure 3: Angular flux in a 2D pin cell lattice geometry, common in light water reactors. Note the high degree of discontinuity, which necessitates a high number of angular samples in order to capture the flux accurately. This sample shows the fast group flux.

3. The Random Ray Method

The Random Ray Method (TRRM) is a new algorithm based on MOC which uses a different strategy for selecting which characteristic lines to solve through the domain of the problem. Instead of selecting a deterministic set of cyclical tracks to cover the domain as shown in Figure 2, tracks are stochastically sampled from a uniform random distribution in space and angle as shown in Figure 4. These rays are followed through the system until termination criteria are met.

TRRM offers a potential solution to many of the issues with scaling up traditional deterministic 2D MOC simulation algorithms used for neutron transport into 3D space. This section provides an overview of TRRM in the context of neutron transport, a discussion of key ideas inherent to the algorithm, and a few demonstrations of these ideas using a reference implementation on a benchmark pin cell problem.

3.1. Reference TRRM Implementation

In the course of our description and presentation of the TRRM algorithm, we utilize a reference implementation in order to perform a number of tests to assess the algorithm's characteristics. This reference application implements 2D and 3D TRRM in high performance multi-threaded C code using OpenMP.

For the purpose of comparison to traditional (deterministic) MOC methods, TRRM will be compared against an open source 2D deterministic MOC application called OpenMOC[2]. Acceleration techniques, such as coarse mesh finite difference (CMFD), that are available in OpenMOC were not used in this analysis as they are not yet implemented in our reference TRRM implementation. As techniques such as CMFD are expected to provide the same magnitude of speedup as in TRRM as compared to traditional methods, the comparison of un-accelerated techniques provided in this paper is likely to be representative of the convergence and accuracy properties when both methods use acceleration.

3.2. Test Problem

Over the course of the following description and discussion of the TRRM algorithm, a 2D nuclear reactor pin cell test case was used in order to provide a baseline for comparison between TRRM and traditional MOC methods. The test case used is the CASL VERA Core Physics Benchmark Problem number 1E[25], which is a standard pressurized water reactor (PWR) pin cell that features an integral fuel burnable absorber (IFBA). The IFBA is a very thin (0.001 cm) coating on the fuel pellet which is made of a highly neutron-absorbing material. As the IFBA is very thin yet has a large effect on the eigenvalue of the problem, traditional methods must use very fine quadratures (i.e., high ray densities) in order to accurately model the problem. This benchmark problem therefore offers maximal insight into how moving from a deterministic quadrature to a randomized one impacts ray density and convergence characteristics. Furthermore, the test problem should provide accurate insight for much larger full core 3D reactor problems, as IFBA pin cells are present in many next-generation reactor designs. While a full core computation may only have several thousand of its approximately 40,000 pin cells as IFBA pins, the quadrature fineness must be uniform throughout when using traditional MOC based methods, so fineness requirements for the IFBA pin cell will need to be applied to the full problem to achieve a high fidelity solution. Thus, ray density and convergence characteristics for an IFBA pin cell should be similar to those for a full reactor core computation.

Additionally, there is previous work by Walker[26] on this topic for traditional deterministic methods that shows clearly that the fineness issue cannot be worked around by way of volume correction or by "smearing" the IFBA material into the gap region of the pin cell without significantly reducing the accuracy of the computation[26]. In addition, to determine the effect of error on IFBA pin cells on full core computations, Walker ran a full AP1000 core midplane and found quadrature errors of much greater than 100 pcm for track spacings below 0.05 cm[26]. Units of *pcm* are used, which stand for "per cent mille" or 1 in 100,000, as it is the common unit for measuring error in reactor physics. If quadrature error is required to be below 50 pcm to ensure acceptable accuracy levels for high fidelity full core computations, running at the standard 0.05 cm ray spacing would not be acceptable even when only relatively few IFBA rods are present in a full core. Work by Walker[26] also provided verification that results produced by OpenMOC on this reference problem accurately represent fixed quadrature based MOC algorithms in general.

While the CASL benchmark problem 1E is well defined in geometry and material composition, there is no multi-group cross section data given. An open source Monte Carlo neutronics code (OpenMC [27]) was used to generate 64 group cross section data for each material of the pin cell. 1 billion particles were run in total, spread over 2000 batches with 400 inactive batches used. This cross section data set was used by both our reference TRRM implementation as well as input to OpenMOC.

For both our TRRM implementation and OpenMOC, the benchmark problem was subdivided into finer flat source regions in order to improve the accuracy of the computation. Specifically, both the fuel and moderator regions were split up into five radial rings each. No additional flat source regions were added to the IFBA, gap, or clad. In total, this created 13 flat source regions for our test problem.

3.3. Algorithm Overview

Now that we have introduced our reference implementation and benchmark problem, we discuss the core ideas and advantages of The Random Ray Method (TRRM).

At a high level, TRRM can be thought of as a hybrid between Method of Characteristics (MOC) and Monte Carlo (MC) methods. Much of the formulation of the algorithm (multi-group approximation, solution along characteristic tracks, use of source regions) is the same as in legacy MOC

methods. The primary departure is in how the characteristic tracks are selected for integration, or more specifically, how the continuous angular and spatial dependence of the MOC equation is discretized.

In TRRM, rather than using a static quadrature of tracks distributed in angle and space deterministically, characteristic origins and angles are picked randomly, as illustrated in Figure 4. The immediate advantages of the method are:

1. Does not require any symmetry or axial homogeneity in geometry. Can work with any 3D CSG geometry.
2. Removes the need to compute and store cyclical tracking and segmentation information, reducing the memory requirements and code complexity.
3. Does not require boundary angular flux to be stored between power iterations, also greatly reducing memory requirements.
4. As the quadrature is changed randomly for each stage of power iteration, the accumulated fineness of the quadrature grows each power iteration – potentially allowing for coarser ray densities to be used when compared to traditional fixed quadrature methods.

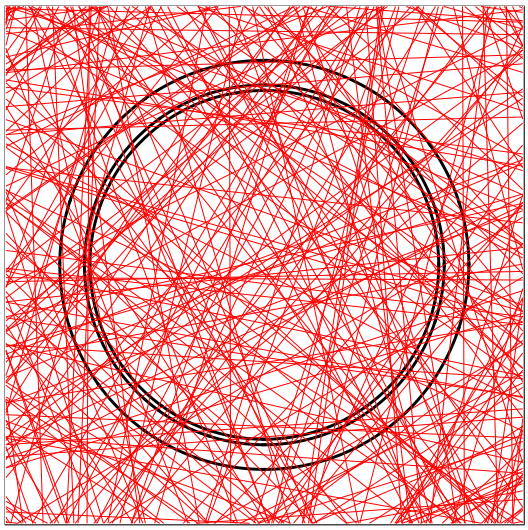


Figure 4: Example of random rays through a pin cell geometry.

The fundamental idea of The Random Ray Method is that randomly selected characteristic tracks are processed using on-the-fly ray tracing through

the constructive solid geometry definition of the reactor. These randomly selected rays form a stochastic quadrature, replacing the deterministic cyclical quadrature used by traditional methods, as shown in Figure 2. Another way of thinking about this is if we considered the reactor to be a multi-dimensional function that is traditionally integrated using the rectangle rule, but we are now instead integrating it via Monte Carlo random sampling.

In TRRM, randomly sampled rays are followed (reflecting through the geometry), solving the characteristic form of the neutron transport equation (Equation 7) as they go, until specified termination conditions are met. After the specified number of rays have been processed, the computation is halted for power iteration in order to update the source regions before repeating.

Traditional MOC quadratures require that various weights be assigned to angles and tracks in order for the total sampled area of the domain to be correct. As an arbitrary number of tracks in TRRM can be used to form the stochastic quadrature, fundamental changes to the weighting scheme are necessary. The volume of each CSG cell, also referred to as a flat source region (FSR), r, is determined in a different way from traditional MOC codes. As tracks no longer have an "area" associated with them as a result of the azimuthal quadrature, the volume of each cell is computed on-the-fly during the neutronics transport sweep by tallying both the total distance travelled by all rays and the distance travelled by rays in each cell r. Upon completion, volumes are calculated as the distance travelled in a cell divided by the total distance that all rays travelled as shown in Equation 8. These volumes can then be scaled, if desired, by the analytical total volume of the bounding box or surface of the domain.

$$V_r = \frac{d_r}{D_{\text{total}}} \tag{8}$$

TRRM utilizes a standard power iteration scheme similar to what is found in most traditional deterministic MOC codes, as shown in Algorithm 1. Notable is the addition of a normalization step following the transport sweep, wherein the scalar flux is normalized to the total distance travelled by all rays during that power iteration's transport sweep. This step must be performed in order to allow for arbitrary numbers and lengths of rays to be used, as well as for correct addition of the scattering and fission sources to the scalar flux. Equation 9 computes the source at each power iteration, where $Q_{r,e}$ is the total source in source region cell r and neutron energy group e, $\Sigma_{t,r,e}$ is the total cross section, $S_{r,e}$ is the scattering source, k is the eigenvalue, and $F_{r,e}$

is the fission source. Equation 10 computes the new scalar flux $\phi_{r,e}$ in each region r and energy group e.

Algorithm 1 MOC Power Iteration

```
Initialize Scalar Fluxes to 1.0
while K-effective and Scalar Flux Unconverged do
    Normalize Scalar Flux to Fission Source
    Compute Source (Equation 9)
    Flatten Scalar Flux to Zero
    Transport Sweep (Algorithm 2)
    Normalize Scalar Flux to Sum of Ray Distances
    Add Source to Scalar Flux (Equation 10)
    Calculate K-effective
end while
```

$$Q_{r,e} = \frac{1}{4\pi\Sigma_{t,r,e}} \left[S_{r,e} + \frac{1}{k} F_{r,e} \right] \tag{9}$$

$$\phi_{r,e} = \frac{\phi_{r,e}}{\Sigma_{t,r,e} V_r} + 4\pi Q_{r,e} \tag{10}$$

3.4. Transport via Ray Tracing

The TRRM transport sweep, shown in Algorithm 2, also features several key differences compared to traditional MOC methods. Each ray is treated as an independent task than can be scheduled by the processor independently. In our reference implementation, this is accomplished using the OpenMP threading model with dynamic thread scheduling. This ensures load balancing, as track integration time will vary. Random sampling of the ray's input parameters is then performed using a random number generator to sample the 3D starting location and direction of the ray (equally probable throughout the simulation domain).

Ray tracing is then performed on the CSG definition of the problem, in a manner very similar to ray tracing performed in Monte Carlo neutronics methods. Similar to many Monte Carlo methods, nearest neighbor lists are used to limit ray tracing only to surfaces bounding the ray's current cell. That is, ray tracing to find the nearest neighbor is done only for the several actual neighbor surfaces rather than every surface in the entire reactor definition. A

Algorithm 2 Transport Sweep

```
for all Rays do
    Distance Travelled D = 0
    Generate Randomized Ray (r̂, Ω̂)
    Apply Ray Boundary Flux Condition
    while D < Termination Distance do
        Set Nearest Neighbor distance s = ∞
        for all CSG Cell Neighbors do
            Ray Trace to Find Distance s_n to Neighbor Surface
            if s_n < s then
                s = s_n
            end if
        end for
        Attenuate Segment s (Algorithm 3)
        D = D + s
        Move Ray Forward or Reflect
    end while
end for
```

typical cell in a reactor pin geometry has approximately 3-7 neighbor surfaces, compared to the millions found in a full core geometry. This localized search leads to extreme efficiency gains as compared to ray tracing on millions of surfaces at every step of the transport simulation, effectively reducing the cost of each ray trace by several orders of magnitude. These ray tracing optimizations are responsible for allowing on-the-fly methods such as TRRM to be comparable in performance to table based lookup methods often utilized in traditional MOC codes that pre-compute a large tracking file that is stored in memory. Alternatively, if using lattice geometry, a hierarchical lattice tree structure can be used to find the initial location of the ray within the lattice via modulus operations, which negates the need for nearest neighbor lists.

Once ray tracing has been completed and a nearest neighbor surface selected, the ray is swept forward over the "segment" using Algorithm 3, wherein each energy group of the ray's angular flux is processed by way of solving the analytical flat source characteristic equation along that line. This algorithm is notable for the absence of azimuthal, track, and polar weighting computations found in traditional deterministic MOC algorithms as weighting is no longer needed in TRRM.

Algorithm 3 Attenuate Segment

1: **for all** Energy Groups $g \in G$ **do**
2: $\quad \Delta\psi_g = (\psi_g - Q_{r,g})\left(1 - e^{-\Sigma_{t,r,g}s}\right)$
3: $\quad \phi_{r,g} = \phi_{r,g} + 4\pi\Delta\psi_g$
4: $\quad \psi_g = \psi_g - \Delta\psi_g$
5: **end for**

3.5. Variable Fineness

Key to the usefulness of any simulation method is the ability to vary the computational resolution to suit the problem. In traditional MOC methods, the spatial and angular components are specified via the number of azimuthal angles, azimuthal track spacing, and the number of polar angles to use. In TRRM, a new set of variables is introduced, namely, the number of rays to be run (each ray corresponds to a new angle) and the distance each ray should travel before termination, as specified in Algorithm 2. By nature, sampling the highly discontinuous angular flux distribution shown in Figure 3 is best accomplished using a maximum number of rays. A high ray count therefore allows for maximally fast convergence.

However, when each ray is born, the starting angular flux of the ray is unknown and must be "guessed" using an approximation. There is therefore a minimum distance required in order to eliminate the error from the initial angular flux guess for the ray. This creates a balance between having a high ray count (speed to convergence) and a high distance per ray (accuracy). Optimizing this selection is discussed in the next several subsections.

3.6. Angular Flux Initialization

In MOC power iteration algorithms, the starting angular flux spectrum must be guessed when a new ray is created. Choice of how to initialize the angular fluxes for each ray is particularly important, as a poor choice may affect the accuracy or required runtime of the calculation. In traditional MOC codes, this guess occurs only once at the beginning of the first power iteration, and from then on the cyclical tracks allow for continuation of all angular fluxes. In TRRM however, angular fluxes are always unknown for each new ray, for all power iterations, greatly magnifying the importance of picking input conditions optimally. While a poor guess can be washed out by following the ray through the geometry for a sufficiently long distance, a more accurate guess can reduce the time it takes to overcome the initial error.

We have tested and considered several choices for initializing the angular flux spectrum for each new ray:

- Vacuum - Start all new rays with 0 angular flux in all energy groups.
- Semi-White – Start new rays with the angular flux distribution of an arbitrary ray that had been previously killed. This is akin to having a reflective boundary condition that (only very rarely) reflects the flux into a random direction rather than the correct one.
- Previous Average – In each power iteration, the average ending angular fluxes for each ray are accumulated and an average flux spectrum for the power iteration is computed. This average spectrum is used as input for all new rays in the next power iteration.

The above methods were all tested using our reference TRRM implementation in order to assess their impact on accuracy. These tests were performed on the 2D IFBA pin cell benchmark problem. All tests were performed with the same number of rays and convergence criteria, while varying the distance each ray travels. Figure 5 shows the error each method generates, in terms of the error in k-effective. Error is measured relative to the asymptotic, extremely high ray density reference solution generated by TRRM. The convergence criteria used to generate this plot will be discussed later in subsection 3.8. As is shown in Figure 5, the "previous average" technique performs the best, offering two orders of magnitude increase in accuracy compared to initializing all rays with zero flux, and an order of magnitude increase in accuracy compared to the "Semi-White" approximation. The remainder of the tests and results in this paper are therefore performed using the "previous average" technique.

3.7. Dead Zone

In the previous section, it was shown that using the "previous average" technique for specifying each new ray's angular flux spectrum allowed for rays to be run accurately at much lower lengths. This allows for much shorter rays to be run to achieve the same overall distance travelled by all rays combined, thus increasing the number of angular samples. However, further optimizations are possible by allowing for a "dead zone" period at the beginning of each ray. This "dead zone" follows the ray through its normal ray tracing and attenuation procedures, but does not contribute back to the

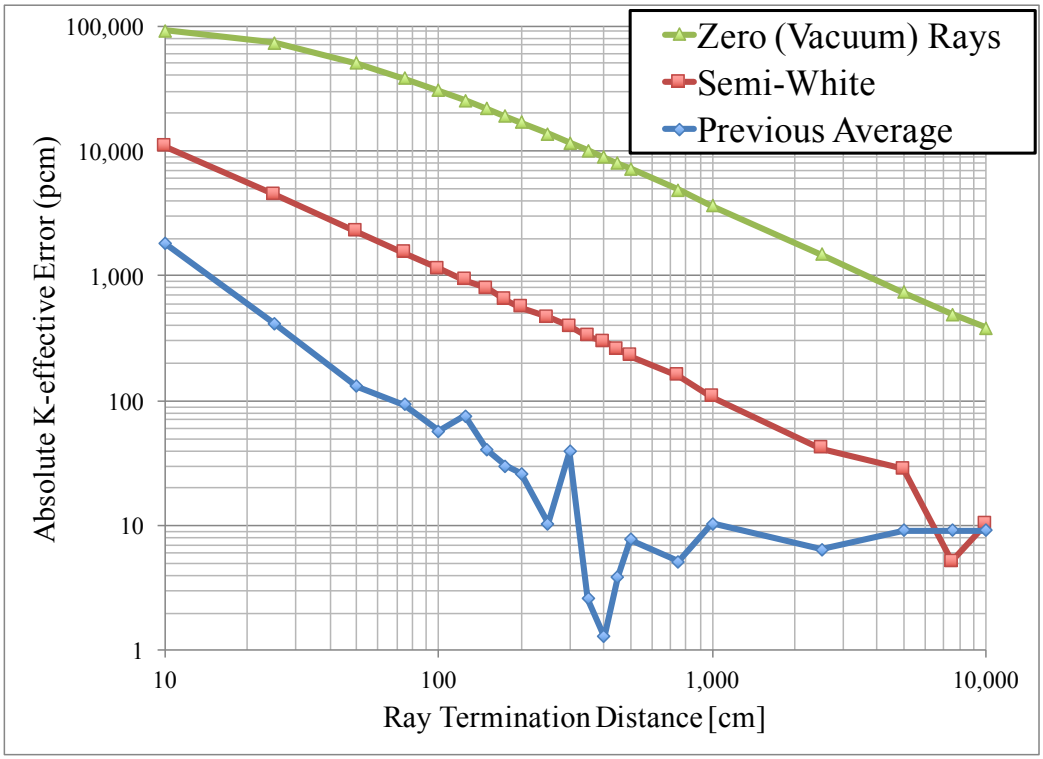


Figure 5: Comparison of several starting ray angular flux estimation techniques. This plot shows the overall error in the computation, measured as the error in the eigenvalue compared to the distance each ray travels.

scalar fluxes of the system. In this sense, the ray is in "read only" mode, which allows for it to alter its initial flux spectrum to something much more accurate while not altering or biasing the system in any way.

It is important to note that time spent in each ray's dead zone is in a sense wasted time, as it is not contributing to the scalar flux solution. Therefore, it is optimal to have a short dead zone compared to the overall length of the ray to minimize this wasted time. To study the balance between dead zone length and increases in overall eigenvalue accuracy, a study was performed for several ray lengths while varying the dead zone distance on a 2D pin cell, as shown in Figure 6. A configuration of 100 cm ray distance with a 10 cm dead zone appears to offer an optimal balance of accuracy, achieving less than 10 pcm eigenvalue error on this test problem, with the dead zone only accounting for 10% of the ray length. Therefore, this is the standard

configuration used for further tests in this paper unless otherwise noted.

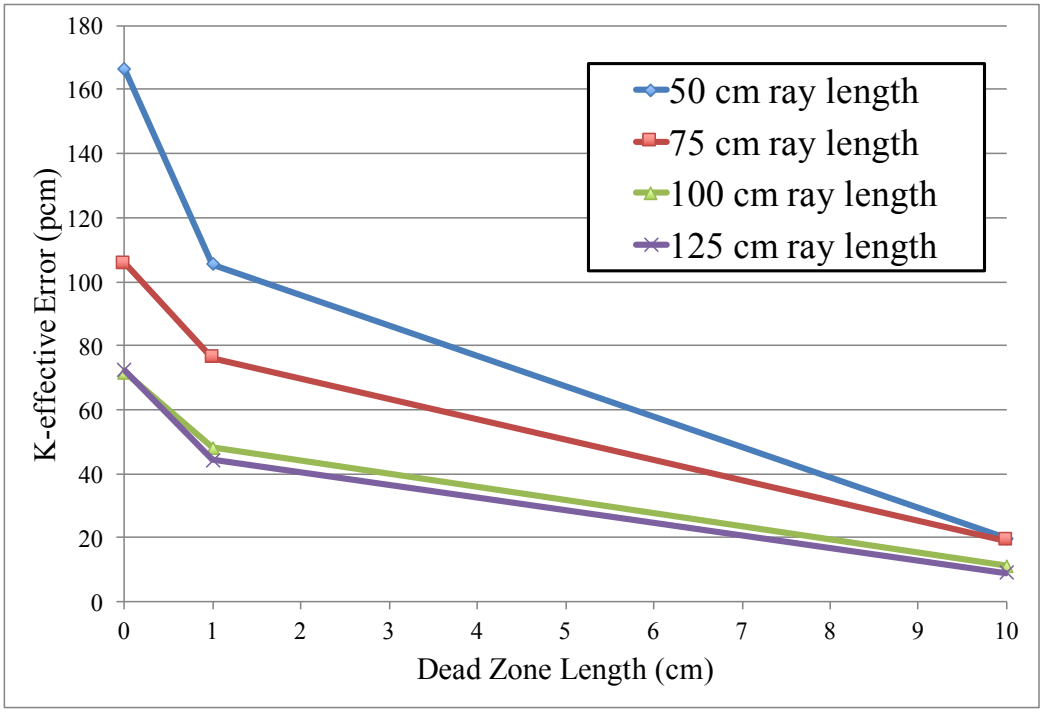


Figure 6: Impacts of various dead zone lengths on overall accuracy of the computation, as measured by error in the eigenvalue.

3.8. Convergence Criteria

As TRRM is a stochastic process, the convergence characteristics of the algorithm are not as straightforward as traditional deterministic MOC methods. In traditional deterministic methods, a root mean squared (RMS) criteria on the scalar flux data array between power iterations is used to determine convergence. In this context, RMS is defined as the entry-wise change in the scalar flux vector (as defined in Equation 11), which ensures that all FSRs and energy groups carry an equal weight in the RMS computation regardless of relative magnitudes. This metric is commonly used as the eigenvalue often converges before the scalar flux, so it is more accurate to measure convergence of the scalar flux than convergence of the eigenvalue itself. Comparing RMS of the scalar flux in this manner between successive power iterations to detect convergence is not practical for TRRM, as the stochastic

algorithm behaves in a similar fashion to Monte Carlo for eigenvalue problems, converging rapidly to the zone of an eigenvalue then producing eigenvalue samples that are normally distributed around the asymptotic solution. While the true power iteration k-effective and scalar fluxes will never converge, the mean of these samples will converge. Therefore, in TRRM it is necessary to accumulate a mean of the eigenvalue and scalar flux array and to measure RMS of this average scalar flux to detect convergence.

$$RMS = \sqrt{\frac{\sum_i^N \left(\frac{\phi_{new}[i]-\phi_{old}[i]}{\phi_{new}[i]}\right)^2}{N}} \tag{11}$$

Similar to Monte Carlo, it is not optimal to begin tracking the mean of k-effective and the scalar flux from the first power iteration[27]. As the starting scalar fluxes and thus source region begin as incorrect guesses, the beginning power iterations are highly inaccurate (as shown in Figure 7). Tracking an average eigenvalue from the beginning means that a huge number of iterations are required to wash out the error of the first series of power iterations, and some bias will always persist in the answer stemming from the starting guesses.

A better strategy is to allow for the scalar fluxes (and thus sources) to converge to a reasonable level before beginning to accumulate the mean k-effective and scalar fluxes from power iterations. These first iterations, akin to "inactive batches" in Monte Carlo[27], could be specified explicitly (e.g., simply run 100 power iterations before tracking the mean), or via automatic convergence detection methods. In our reference TRRM implementation, a basic windowing method has been implemented such that the mean eigenvalue is tracked along a travelling window (e.g., the previous 25 power iterations) until a specified standard deviation of the eigenvalue is met (e.g., ± 25 pcm). At this point, the "active region" is entered and k-effective is accumulated continuously (without windowing), along with an average scalar flux, until the RMS of the average scalar flux converges (e.g., to below 1E-5). An example of this process, as run on a 2D pin cell, is shown in Figure 7 and Figure 8.

It is worth noting that there are many possibilities for new acceleration techniques aimed at reducing the number of iterations required for convergence. For example, since an arbitrary number of tracks can be run during each iteration, it may be beneficial to run few tracks (a coarse problem) for the first set of power iterations before increasing the number of tracks (a fine problem) as the power iteration count rises. Additionally, traditional MOC acceleration

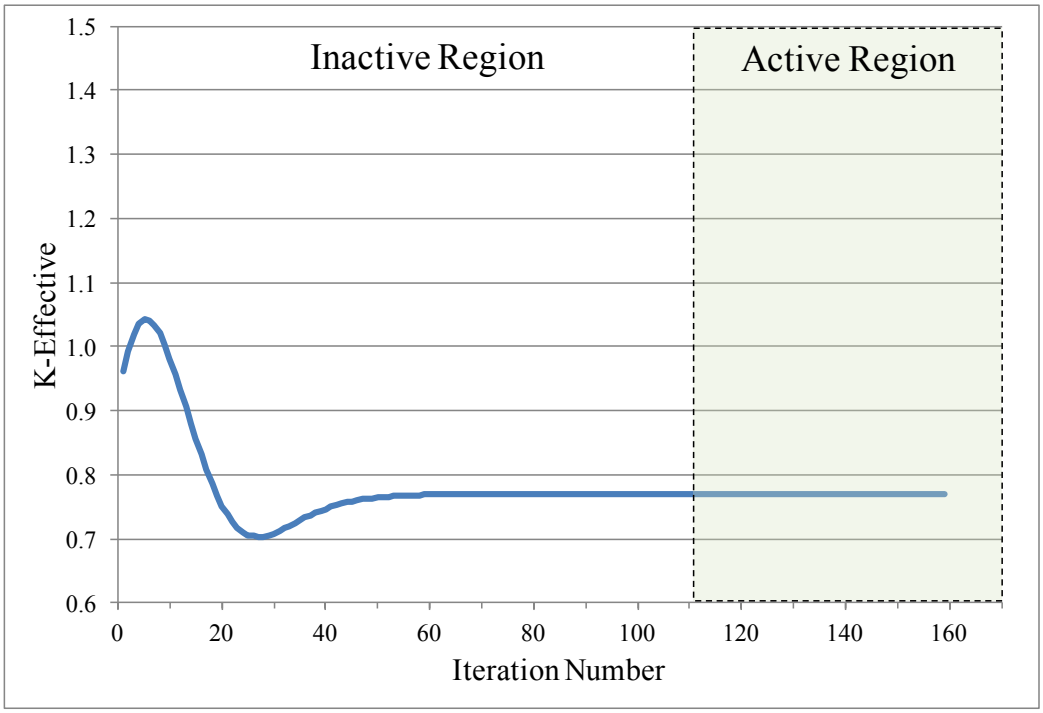


Figure 7: Example of TRRM convergence on a 2D pin cell problem. The left region is "inactive," following the normal power iteration procedure. Once travelling window convergence criteria are met, the "active" region begins wherein a mean k-effective is accumulated.

schemes such as coarse mesh finite difference (CMFD) methods that utilize a diffusion solve in between high fidelity MOC transport solves should also be compatible with TRRM. CMFD is usually able to reduce the number of power iterations required for convergence by a factor of 5-10x. It is expected that implementation of CMFD should be of the same complexity and difficulty as compared to usage with traditional deterministic methods. It is important to note, however, that CMFD is not guaranteed to be numerically stable, so the stochastic nature of TRRM may make practical usage and tuning of CMFD more difficult. The numerical instability of CMFD is expected to be a tractable problem though as CMFD has been used successfully in Monte Carlo reactor simulations[28].

Alternatively, TRRM can also be configured to generate rays using the same random number seed for each power iteration. This has the effect of

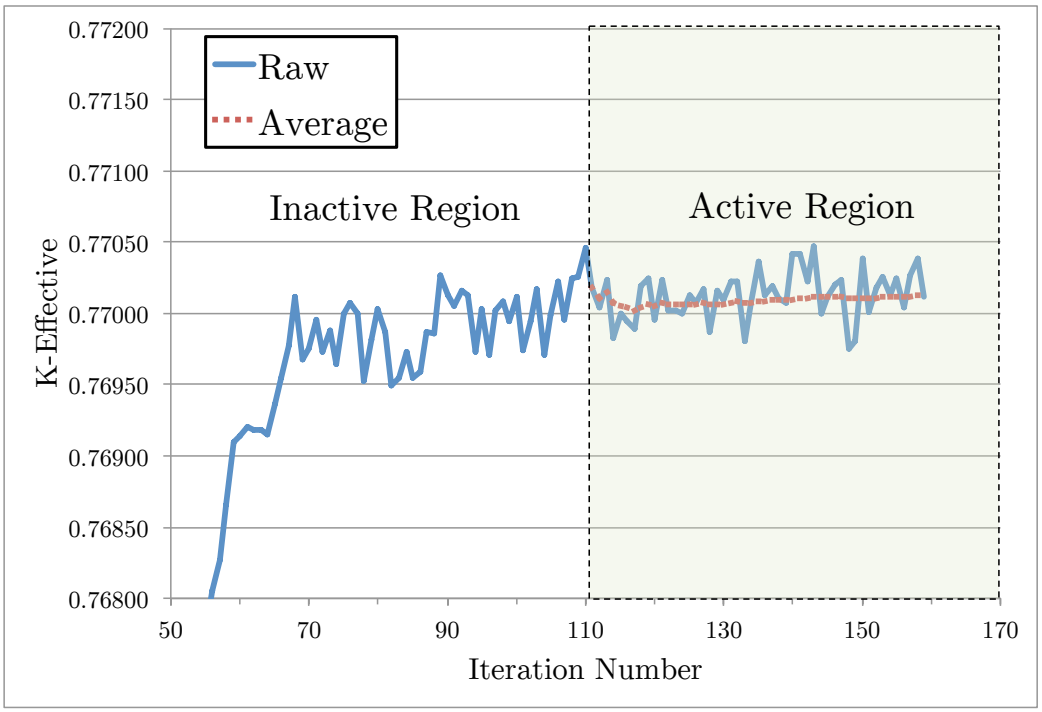


Figure 8: This is a zoomed in view of Figure 7, which is framed to highlight the approach to convergence and the entering of the active region. This close-up view also shows the accumulated mean eigenvalue in the active region.

allowing the same quadrature to be used over all power iterations, therefore converging in a manner identical to traditional deterministic MOC methods. This method, known as "procedural" quadrature generation, may be beneficial for some usage scenarios where unstable acceleration techniques are incompatible with the stochastic convergence properties of normal TRRM. However, procedural generation removes some of TRRM's coarseness advantage as the resolution must be fine enough to resolve the problem accurately without accumulating samples over power iterations. Procedural generation also removes the ability to easily alter the number of rays or ray lengths between power iterations. However, procedural TRRM is still useful in the case where there is extremely fine detail in the geometry that causes significant coarseness errors for evenly spaced (traditional) tracks. In such cases, the randomized quadrature may produce more accurate results with fewer rays. For all results presented in this paper, the normal TRRM method is used

wherein the random number seed changes between power iterations resulting in stochastic convergence properties, as shown in Figure 8.

4. Preliminary Performance Testing and Analysis

4.1. Overview

While the upsides of The Random Ray Method are numerous and clear (e.g., full geometric flexibility, greatly reduced memory requirements, no added memory overhead for 3D, simplified code base, sparser ray density requirements, etc.) there exist three central questions as to the usefulness of the algorithm. Namely:

1. **Ray Tracing Overhead** – Now that ray tracing must be performed on the fly for every geometric intersection, what are the costs as compared to traditional methods where this is precomputed and read out of memory?
2. **Convergence** – In the conversion of the deterministic MOC algorithm to a hybrid deterministic-stochastic process, what is the impact on convergence rate? How many power iterations are required to converge the eigenvalue and flux as compared to traditional methods? Combined with item (1), what does the time-to-solution look like?
3. **Accuracy** – Is TRRM able to provide the same level of accuracy as traditional MOC methods?

In this section, a number of tests aimed at providing answers to the above three core questions are performed. In order to isolate the differences between TRRM and traditional methods (i.e., OpenMOC), all test cases unless otherwise noted were performed in 2D using a single polar angle. Our implementation of TRRM is native to 3D, so a reduction in dimensionality to 2D is achieved by changing the way new rays are sampled. In 2D, the polar angle for a new ray is taken from the single angle TY polar quadrature[29], rather than freely sampled in space (i.e., random cosine between -1 and 1). The problem is still modelled and ray-traced in 3D with the addition of top and bottom reflective boundary conditions specified so as to be a fully reflective pin cell cube.

This reduction to 2D was done to offer an "apples to apples" comparison to the deterministic 2D OpenMOC code and previous work on the topic[26].

4.2. Performance

4.2.1. Time per Intersection

In order to estimate the performance impact of doing on-the-fly ray tracing, performance was compared between TRRM and the traditional deterministic MOC implementation (OpenMOC) directly via a common figure of merit – the amount of time it takes to attenuate a single energy group of flux across a single geometric segment (i.e., the time per *intersection*, or time per integration). All tests in this analysis were performed on a dual socket Intel Xeon E5-2650 "Sandy Bridge" CPU node (16 cores, 32 threads total). In the case of TRRM, this metric also includes the time spent performing on-the-fly ray tracing.

Results from running the 2D IFBA pin cell benchmark test case (with 64 energy groups) for both methods are given in Figure 9. As can be seen, the fully 3D TRRM implementation runs 2.3x faster in serial (and 14.2x faster in parallel) than the 2D deterministic implementation of OpenMOC.

An additional test of the TRRM implementation was performed on a newer system, a dual socket Intel Xeon E5-2699 v3 node (36 cores, 72 threads). This node produced a serial integration time of 2.69 ns per intersection (and 0.0871 ns per intersection in parallel). While serial integration time is often fairly constant for a code between systems, the speedup here is likely due to the highly vectorized nature of the TRRM algorithm. As the inner loop is over energy groups, of which there are 64 in this problem, the code can make full use of the wider "Same Instruction Multiple Data" (SIMD) vector widths of the Haswell architecture as compared to what is available on the older Sandy Bridge processors. The vector widths are twice as wide on Haswell, and the results between the two architectures from our code mirror this closely. This is an important result as vector widths are continuously widening with each new architecture generation meaning that the strong vectorizability of the TRRM algorithm ensures that it will continue to run extremely fast on next generation hardware architectures[12]. [1]

[1] It is important to note the excellent SIMD performance of TRRM is also shared by deterministic MOC based methods in general, though is not shared with Monte Carlo. In Monte Carlo, particles are generally not stored and vector processed together due to each particle sampling from a variety of events and requiring different code be executed. "Event based" formulations of Monte Carlo require that particles be halted and sorted at each computation step, which may allow vectorization but at the cost of significantly increased memory movement which in practice is often worse than loss of vectorization.

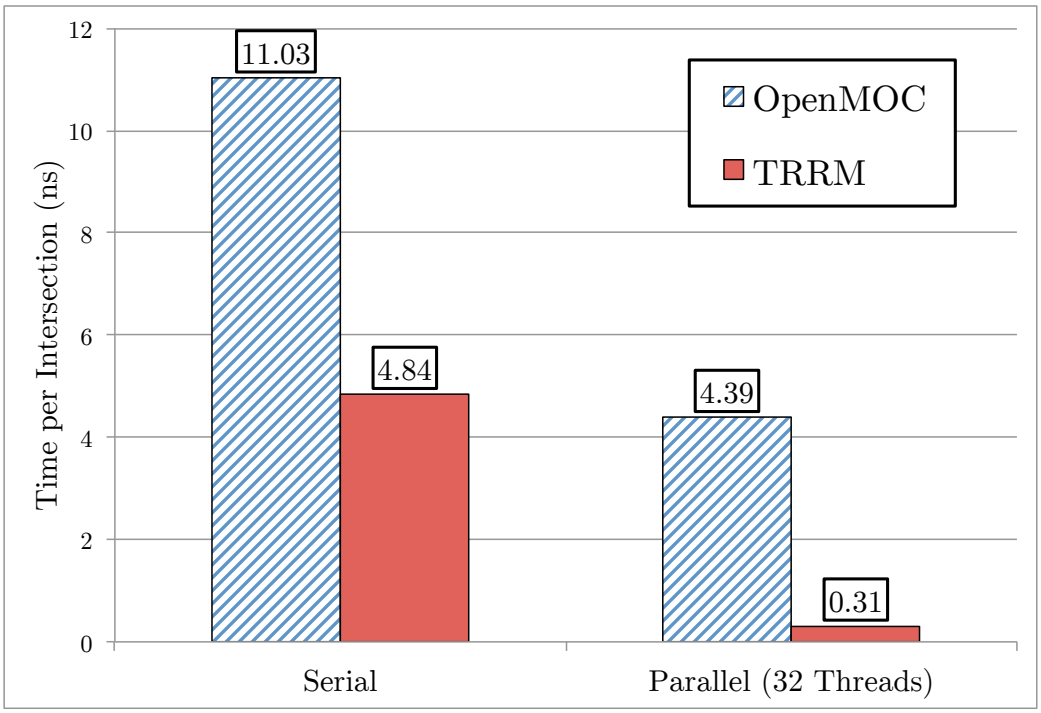


Figure 9: Comparison of the MOC performance "figure of merit" (time per intersection, in nanoseconds) for a 2D IFBA pin cell between TRRM and traditional deterministic MOC methods (as implemented in OpenMOC) for 64 energy groups. For collecting data for this run, the traditional MOC method is running 128 azimuthal angles with 0.001 cm track spacing.

As an additional point of reference, a time of 10 ns per intersection (serial) is considered a standard benchmark for a highly optimized production 2D MOC code. Even compared with this more aggressive target, TRRM performs 3.7x faster at only 2.69 ns per intersection on the Haswell system.

In TRRM, a single ray trace is performed per geometric intersection, with all energy groups then having to be integrated over the interval (i.e., having a larger number of energy groups amortizes the costs of ray tracing). To get

While TRRM is also stochastic, the same operations are always being performed on rays, and vectorization occurs within the ray across energy groups rather than on a higher level over rays, so it affords a high degree of vectorizability provided a sufficient number of energy groups are used.

a sense of how the costs of ray tracing impact integration time for a variety of real world scenarios, the program was run for a single power iteration both normally and with the core flux attenuation function removed. These results, shown in Figure 10, reveal that about 34% of the runtime is spent ray tracing when 64 energy groups are used. When only 8 energy groups are used, time spent ray tracing increases to 60%. The overall increase in integration time seen when only 8 groups are used is a result of both the increased relative cost of ray tracing as well as the smaller SIMD vector width reducing the efficiency gains from vectorization.

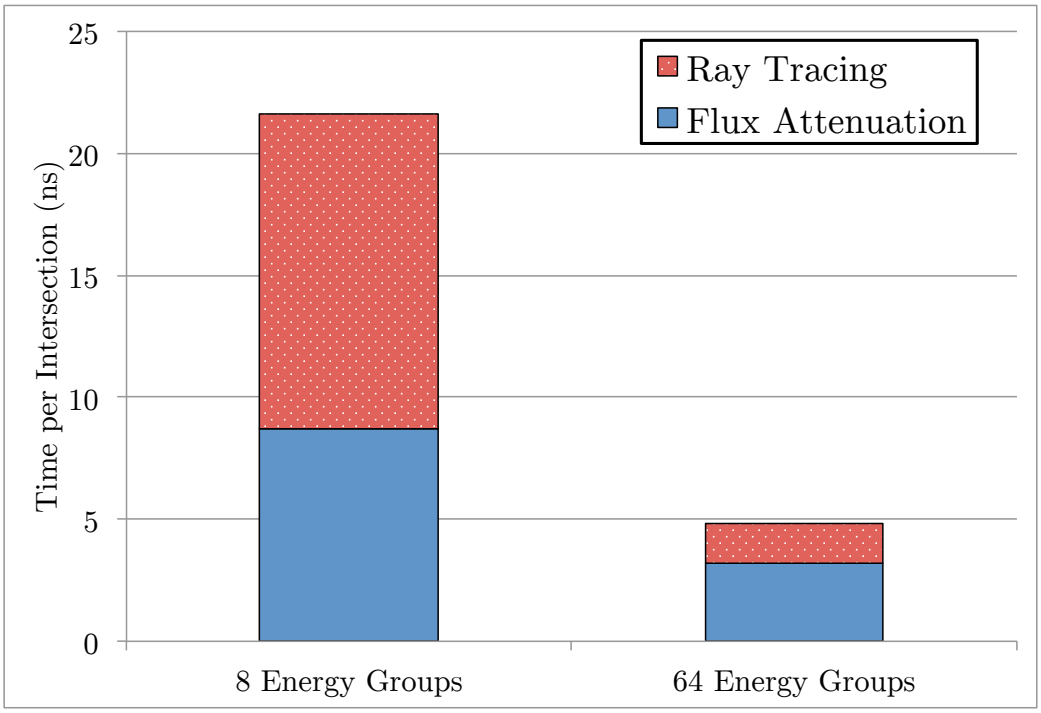


Figure 10: Breakdown of ray tracing costs versus integration costs for TRRM.

These results point to a performance weakness of the algorithm in that it is not well suited for extremely low fidelity computations that only use a minimal number of energy groups. A two group TRRM computation may be several times slower (in terms of time per intersection) than a traditional deterministic method which does not need to perform ray tracing on-the-fly. However, the trend in the field towards higher fidelity computations required by advanced reactor designs necessitates use of many energy groups. As

more energy groups are used, the on-the-fly ray tracing costs become trivial, which means that the algorithm will scale extremely well for medium and high fidelity computations that feature 10's or 100's of energy groups.

4.2.2. Memory Efficiency

Another important metric for scalability is memory efficiency. For the 2D IFBA pin cell benchmark (with 64 energy groups), we calculated the amount of memory required for each of the methods by summing the significant memory allocations for each of the programs. The numbers for OpenMOC were taken for a 128 azimuthal angle and 0.001 cm ray spacing configuration with 1 polar angle, which resulted in 102,756 tracks and 1,189,908 segments requiring explicit storage in memory. The TRRM implementation's memory usage does not vary based on the number of rays or termination distances, only requiring the basic XS data and CSG information. These results are shown in Table 1.

Table 1: Memory usage comparison of TRRM and OpenMOC for a 2D IFBA pin cell problem

	TRRM	OpenMOC
Boundary Angular Flux [MB]	-	100.35
Track and Segment Data [MB]	-	42.75
CSG & Nearest Neighbor Lists [MB]	0.0058	-
Scalar Flux [MB]	0.044	0.025
XS Data [MB]	0.17	0.17
Total [MB]	0.2198	143.3

The TRRM implementation requires about 53 KB in total, a memory savings of over 652x compared to OpenMOC which used 143.15 MB. These extreme memory savings can be attributed partially to the fact that TRRM does not require storage of a large pre-computed ray tracking and segmentation file as is used in deterministic applications like OpenMOC. More importantly, TRRM does not require storage of angular boundary fluxes between iterations, instead making use of on-the-fly ray initialization techniques (described in subsection 3.6 and subsection 3.7) to remove entirely this memory requirement. This extreme increase in memory efficiency allows for great increases in scalability by allowing for much larger problem sizes to be run on a given

computer or cluster. Importantly, this 652x increase in memory efficiency also aides CPU performance significantly by allowing for much greater cache efficiency and reducing (or in this case, negating) the need to wait for data to arrive from main memory, which helps to explain the performance results of the previous subsection.

4.3. Asymptotic Accuracy

The previous several sections have shown that TRRM offers a significant performance advantage in terms of time per integration, which results from the greatly improved memory efficiency of the algorithm. However, it is important to determine if TRRM can provide similar levels of accuracy to traditional deterministic MOC methods. In this section, we will compare the highly converged solutions between TRRM and traditional methods when running extremely fine (high ray density) quadratures.

Table 2 compares the eigenvalues for four codes on the CASL VERA Core Physics Benchmark Problem number 1E[25] that is discussed in subsection 3.2. The CE KENO-VI code is the Monte Carlo neutronics code that was used to generate the reference solution to the benchmark problem, with results taken from the benchmark definition[25]. OpenMC is the open source Monte Carlo neutronics code that we used to generate the cross section data for usage in OpenMOC and our TRRM implementation. The eigenvalues generated by CE KENO-VI and OpenMC differ by 99.6 pcm, which could be due to differences in the pointwise cross section libraries (or which version) that were used for each computation or in the processing of these libraries. These differences are not important, as the purpose of this paper is to compare accuracy between traditional deterministic based MOC methods and TRRM, so any reasonable set of multi-group cross section data is sufficient for this analysis.

The highly converged eigenvalues for OpenMOC and TRRM only differed by about 3 pcm. This highly converged solution for OpenMOC corresponds to a ray spacing of 0.0005 cm with 128 azimuthal angles and 1 polar angle, using the full exponential intrinsic evaluation. These two methods therefore can be said to produce equivalent answers with small differences caused by using different angular quadratures and spatial ray layouts, showing that the stochastic nature of TRRM does not impart any bias to the solution. These two methods generate solutions that are around 300 pcm below OpenMC, which was the source of their cross section data. This bias of several hundred pcm is expected. While there is always some error when moving from

continuous energy cross section space in Monte Carlo to discrete multi-group cross sections in MOC, the majority of this error is likely due to neither OpenMOC nor our TRRM implementation accounting for anisotropic scattering or the angular dependence of cross sections. These factors can be corrected for in OpenMOC and TRRM in a straight-forward manner using transport corrected cross section data, but were not implemented in TRRM for the sake of simplicity in analysis of the new method. The critical objective of this paper is to compare accuracy and convergence between traditional MOC methods like OpenMOC and TRRM, so implementation of correction routines identical to both methods was not worthwhile for this analysis.

Analyses in the rest of this paper regard accuracy for OpenMOC and TRRM as being compared to each method's asymptotic solution. This is done as there is no basis to claim which code has a more correct solution when their difference is only 3 pcm. Therefore, error is simply relative to each code's own highly converged solution.

Table 2: Comparison of highly converged eigenvalues

	CE KENO-VI	OpenMC	OpenMOC	TRRM
k-effective	0.77169	0.77246	0.77010	0.77012

4.4. Convergence and Accuracy

The previous section showed that TRRM can produce the same highly converged solution as traditional deterministic methods. However, given the stochastic nature of the new algorithm, the convergence rate (in time and accuracy) will be likely to change. This will have effects both in terms of the number of power iterations required and the fineness of the problem required to converge to a solution. In this section, we will use our reference implementation of TRRM and the traditional deterministic MOC code (OpenMOC) to evaluate the convergence characteristics of the two algorithms on the benchmark IFBA pin cell problem.

Traditionally, 2D MOC based methods have typically used a fixed cyclical azimuthal quadrature composed of 128 angles and a track spacing of 0.05 cm[26]. These parameters allow for a high degree of accuracy for simulations where the smallest geometrical elements are the helium gap spacing that exists between the fuel pellet and the cladding in a pin cell geometry. As

the width of the helium gap is approximately 0.05 cm in most designs, this correlates closely with the required track spacing of the computation. As newer reactor designs feature a large array of advanced fuel designs that help flatten the flux profile, extend the cycle time for the reactor, and allow for higher degrees of reactor controllability, the minimum element size required to maintain high fidelity will greatly decrease. As is the case with our benchmark IFBA problem, the IFBA is only 0.001 cm thick (50 times thinner than the helium gap). This 50x decrease in minimum geometrical element size should predict that a traditional fixed quadrature would need to be 50 times finer to maintain fidelity. Additionally, the IFBA material has a much bigger impact on the flux profile due to its highly absorbing nature, compared to the helium gap which is neutron transparent. This increased reactivity may also necessitate a greater fineness than the 0.001 cm thickness predicted.

The IFBA test case was analyzed in-depth by Walker[26] for traditional deterministic quadratures. We have performed a similar study using OpenMOC. We fixed the number of azimuthal quadrature angles at 128 and then refined the track spacing from 0.1 cm incrementally down to 0.00025 cm, as shown in Figure 11. A zoomed in view for the finer range is given in Figure 12. As can be seen, the regular version of OpenMOC produces errors of 2505 pcm in the eigenvalue at the typical 0.05 cm ray spacing. As predicted, the track spacing does not produce accurate results (with "accurate" considered as less than 50 pcm eigenvalue error) until refined to the thickness of the IFBA at 0.001 cm. These results are similar to those reported by Walker[26].

One idea to improve the results of the traditional method is to analytically correct the volume of the IFBA in order to ensure that it is not over or under represented in the overall integration of the problem. This can be done by altering the segment lengths of tracks that pass through the IFBA so that when their lengths and "widths" are summed they correctly add to the analytical IFBA volume. This feature is available in OpenMOC and is listed as the "volume corrected" line in Figure 11 and Figure 12. This feature greatly reduces the error for very coarse runs, for instance bringing the error from 2505 pcm down to 1257 pcm at a spacing of 0.05 cm. However, as the quadrature is further refined, the volume corrected strategy produces errors of similar magnitude as to the uncorrected version. This means that even with a precise analytical volume specified, track spacings of above 0.001 cm still generate large errors in the flux integration. Overall, these results match closely what was found by Walker[26], and show that traditional fixed quadrature based MOC methods require extremely fine quadratures (and

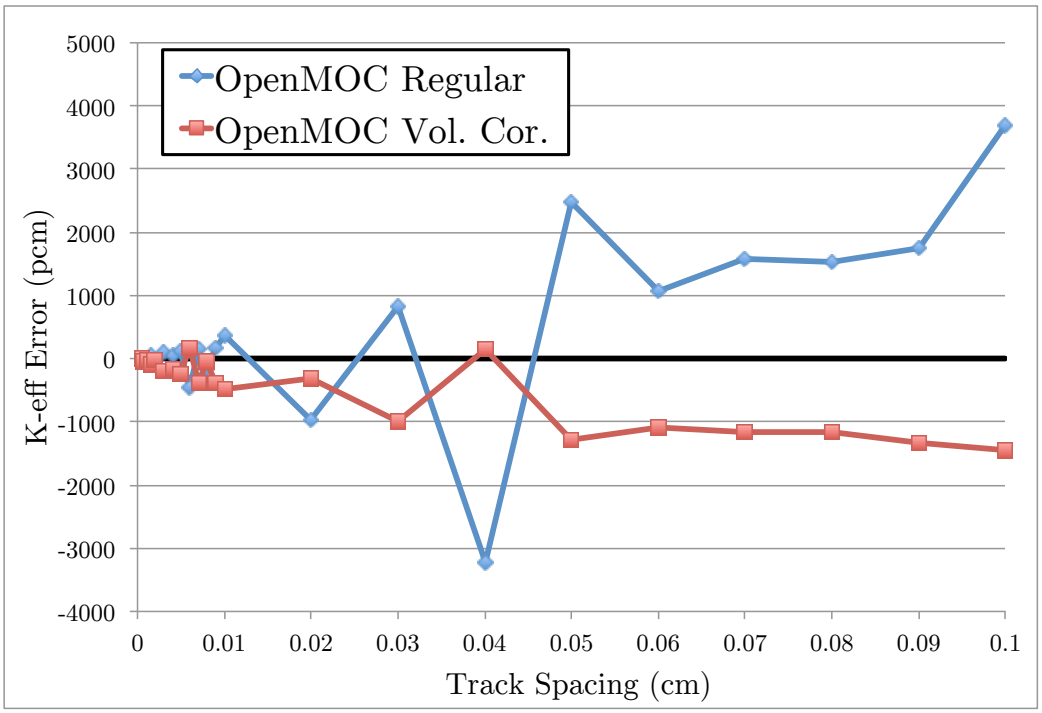


Figure 11: Traditional deterministic methods – eigvenvalue (k-eff) error in pcm (parts per 100,000) versus track spacing for a 2D IFBA pin cell problem. Results were generated using OpenMOC with 128 azimuthal angles.

therefore long runtimes) in order to accurately model reactors featuring IFBA pins.

To determine the difference in required quadrature fineness (ray density) between TRRM and traditional methods, a large number of configurations were run using TRRM to form a basis of comparison. As TRRM is a stochastic process, the resulting eigenvalue from each independent run of the code will be different (when a new random number seed is used each time). To ensure that the numbers produced for each configuration are reliable, the code was run 1000 times for each configuration to determine the mean eigenvalue as well as the true standard deviation of the eigenvalue at that coarseness level. Configurations ranged from running many thousands of rays per power iteration down to running only 5 rays per power iteration. For any configuration used for TRRM in this study, results were always distributed around the method's asymptotic mean in a normal distribution. Even with

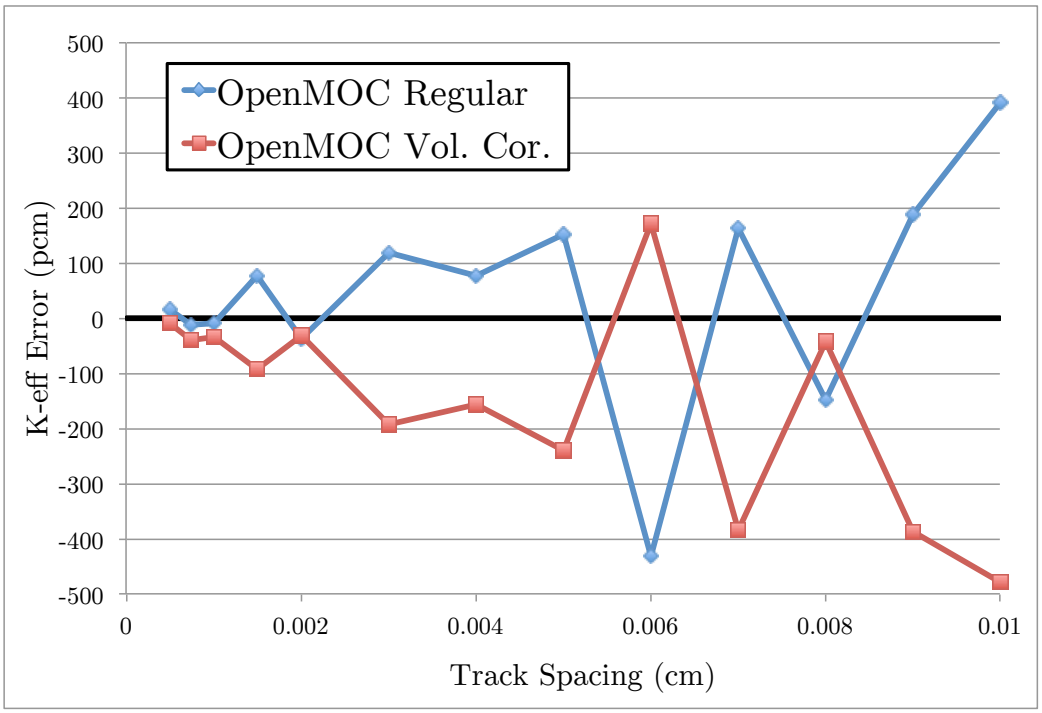


Figure 12: Zoomed in view of Figure 11 showing more clearly the finer track spacing configurations.

extremely coarse (low ray density) configurations, the code would produce eigenvalues normally distributed around the true mean, without bias in one direction or another. Coarser configurations required more power iterations before convergence criteria were met, but still resulted in far fewer total intersections being run.

As there is no concept of track spacing in TRRM, the total number of intersections to converge the solution to 1e-5 root mean square (rms) of the scalar flux array was recorded. An "intersection" is defined as the attenuation (i.e., integration) of a single energy group of flux across a single geometric segment. Data from the 1000 runs for each configuration were tabulated in Figure 13, which shows the average number of intersections versus the statistical distribution of the converged mean eigenvalue. The distribution is displayed by coloring the 1σ, 2σ, and 3σ regions of the distribution, which is an optimal method for displaying uncertainty intervals developed by Aggarwal and Caldwell[30]. This allows for an "apples to apples" comparison with

traditional methods, which are also plotted on Figure 13 but using the total number of intersections metric rather than the track spacing. A zoomed in view showing the finer detail in convergence and accuracy (thus cutting off the larger errors shown in the traditional OpenMOC method) is given in Figure 14.

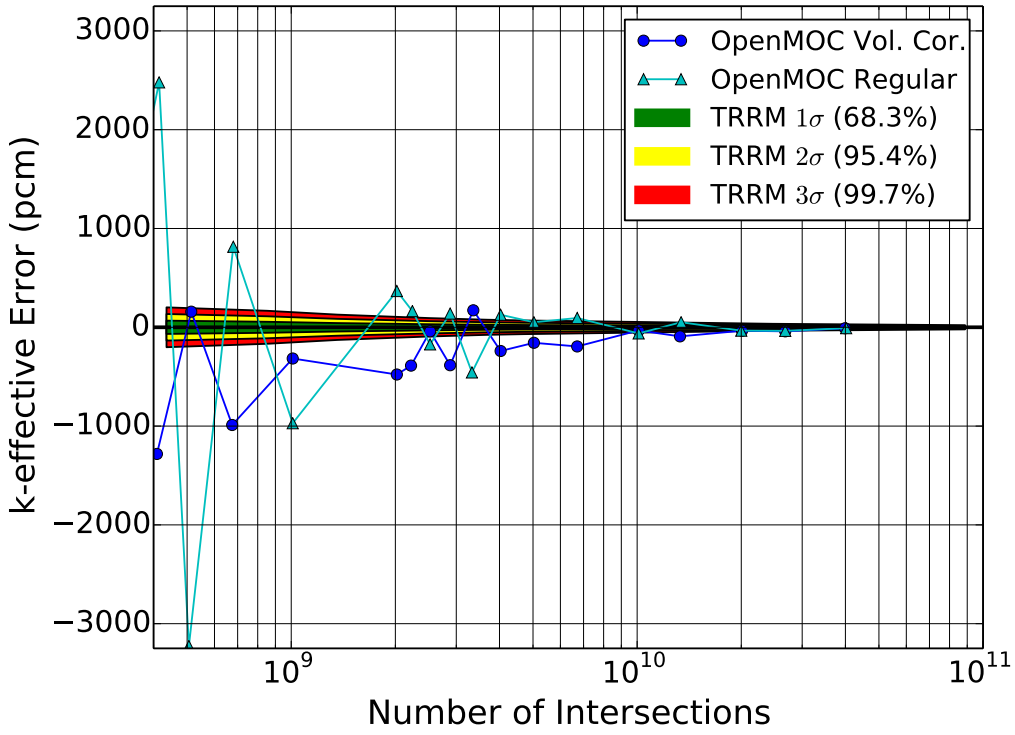


Figure 13: Comparison of TRRM to traditional deterministic methods. This plot shows eigvenvalue (k-eff) error in pcm (parts per 100,000) versus total number of intersections required to converge to within 1e-5 rms of the scalar flux for a 2D IFBA pin cell problem. Deterministic results were generated using OpenMOC with 128 azimuthal angles, both with and without analytical volume correction.

Overall, Figures 13 and 14 show clearly the great difference in the amount of work required to accurately converge the IFBA pin cell problem between the two methods. At the coarsest level measured (4e+8 total intersections) TRRM produces an eigenvalue with a standard deviation of 67.2 pcm. At the 3σ confidence level, this means the error in the eigenvalue will be within 201.7 pcm 99.7% of the time. The fixed quadrature method with volume correction,

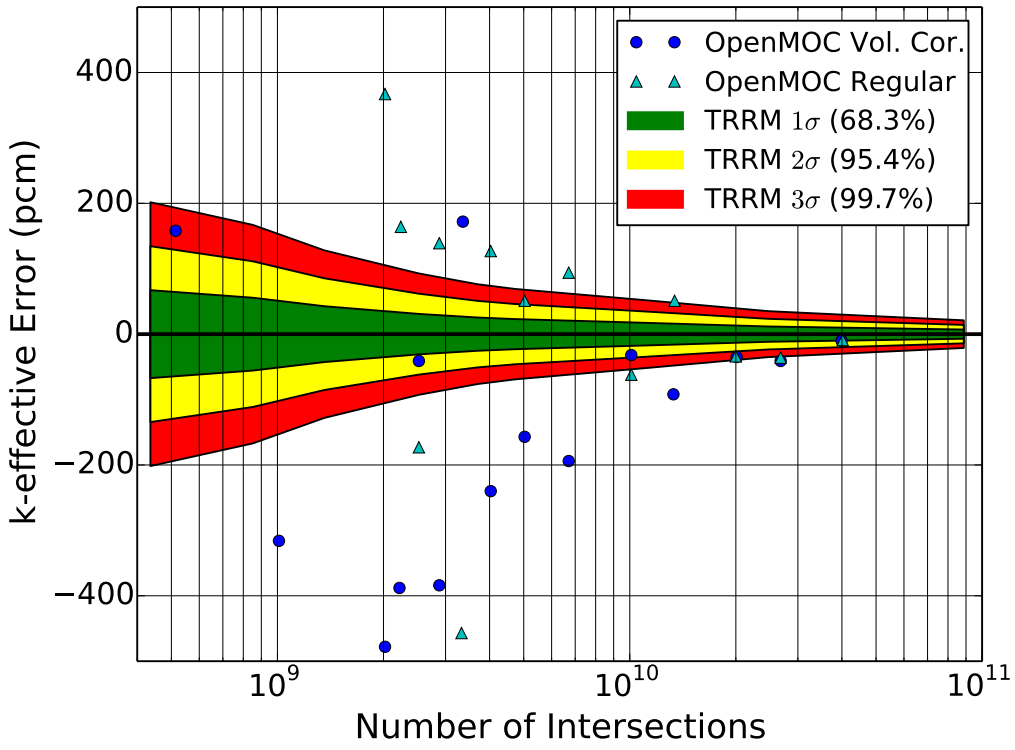


Figure 14: Zoomed in view of Figure 13 showing the fine detail in error for the finer configurations.

for a similar number of intersections, OpenMOC produces an error of 1257 pcm. This means that for equivalent "algorithmic work" (i.e., 4e+8 segment integrations) TRRM is 6.23x more accurate at the high confidence 3σ level.

Another metric to consider is the total number of iterations required to converge a problem to a specified accuracy and confidence level. A number of accuracy and confidence levels are considered in Table 3, which shows the number of intersections required to reach these levels for both methods. In the case of OpenMOC, the 100 pcm error level corresponds to a track spacing of 0.004 cm and 128 azimuthal angles, and the 50 pcm error level corresponds to a track spacing of 0.001 cm with 128 azimuthal angles. Table 4 shows the magnitude of intersection savings that TRRM achieves over traditional methods. A good example from these tables to consider for a real world usage scenario is if we wanted to know the eigenvalue to within 50 pcm quadrature error, and we wanted to know this to within the 2σ level. In

this case, traditional methods would need to perform 5.2x more intersections to achieve the same accuracy as compared to TRRM. Combined with the significant performance speedup in intersection time that TRRM has displayed in Figure 9, one could expect a large reduction in overall runtime by using TRRM instead of fixed quadrature based methods, as will be discussed in the next section.

Table 3: Number of Intersections Required to Achieve Specified Accuracy and Confidences

	TRRM			OpenMOC
	1σ	2σ	3σ	
Within 100 pcm	4.36E+08	1.07E+09	2.28E+09	5.03E+09
Within 50 pcm	1.07E+09	3.88E+09	1.26E+10	2.01E+10

Table 4: Algorithmic performance gains of TRRM over traditional methods, in terms of the ratio of number of intersections required for OpenMOC over TRRM.

	1σ	2σ	3σ
Within 100 pcm	11.5	4.7	2.2
Within 50 pcm	18.6	5.1	1.6

In summary, this section provided a compelling argument for the usefulness of the TRRM algorithm. These results show that TRRM is not only faster in time per intersection, but also affords a significant reduction in the total number of intersections required to accurately resolve an IFBA pin cell problem.

4.5. Integral Performance Testing

The previous subsections have laid out the advantages of TRRM compared to traditional methods in three key areas: performance, memory efficiency, and algorithmic efficiency. In this section, an integral test is performed which measures the overall time to solution for TRRM and OpenMOC on the 2D IFBA pin cell problem. All runs were done using our dual socket Intel Xeon E5-2650 "Sandy Bridge" CPU node (16 cores, 32 threads total). Figures 15 and 16 show that TRRM provides a significant cumulative advantage over traditional deterministic methods.

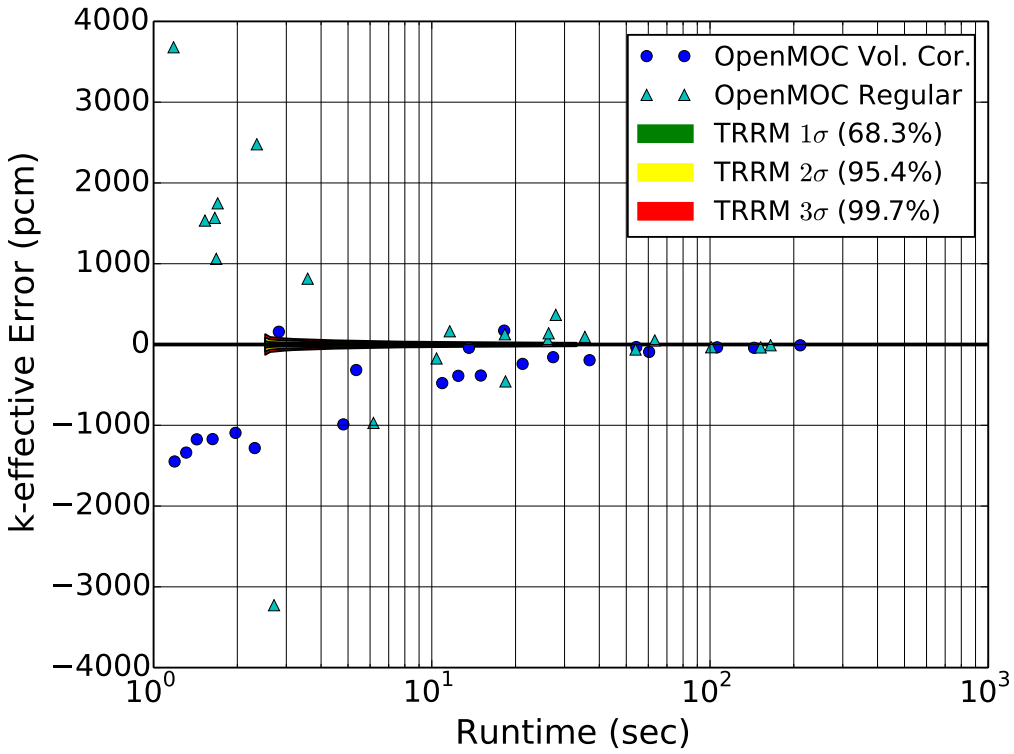


Figure 15: Overall time-to-solution versus accuracy for TRRM and traditional deterministic method (OpenMOC). See Figure 16 for a zoomed in view.

This data can also be analyzed in a similar method to the previous section, selecting particular accuracy requirements and measuring the time required for various confidence levels of TRRM. This analysis is shown in Table 5. Additionally, the overall time-to-solution speedup of TRRM over traditional methods is given in Table 6, which shows the factor by which TRRM is faster than OpenMOC. For instance, these results show that if one wanted to know the eigenvalue to within 50 pcm accuracy with 3σ confidence, TRRM can produce a solution 20.9x faster than traditional methods.

One note to highlight in Table 6 is that several of the data points are the same. This is because the extremely coarse TRRM runs only utilize 5 discrete rays per power iteration. As thread level parallelization occurs over rays, this means only 5 threads out of the 32 available on the system could be used. As more rays are added, the ray density per power iteration increases thus reducing the number of power iterations required to converge

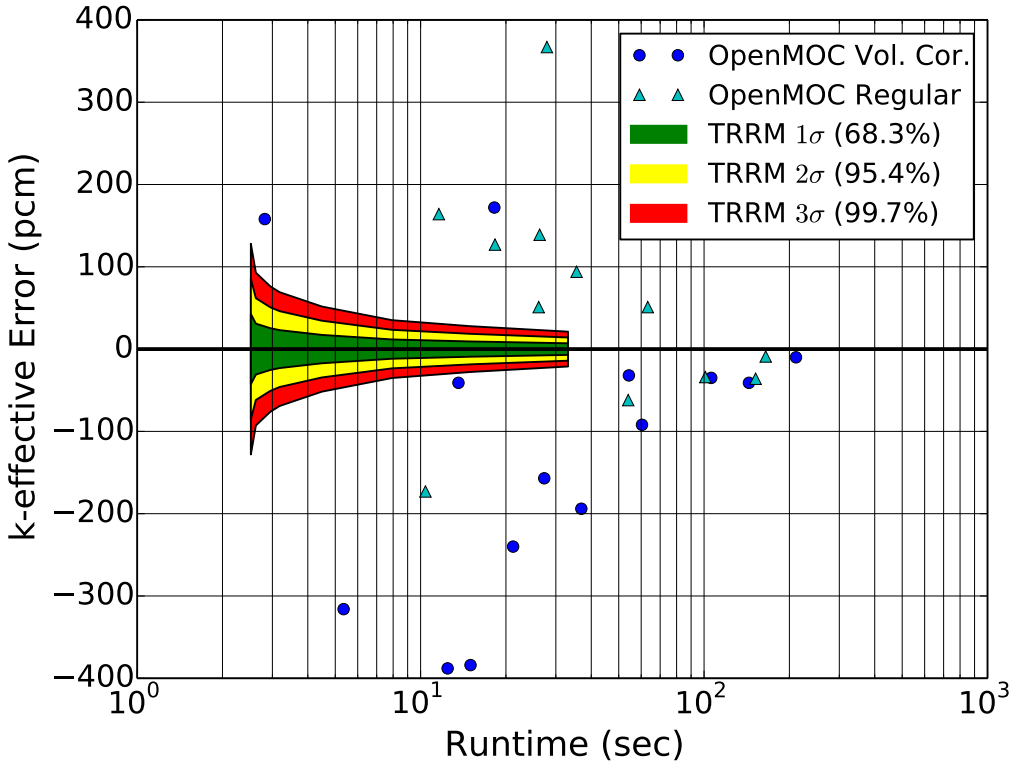


Figure 16: Zoomed in view of Figure 15 showing greater detail in error for finer configurations.

the solution while at the same time adding system resources (threads) to the computation. The net result is that for TRRM when 32 rays were used, the solution had already converged to 100 pcm accuracy with 2σ confidence and 50 pcm accuracy with 1σ.

Table 5: Runtime (sec) required to achieve specified accuracy and confidences

	TRRM			OpenMOC
	1σ	2σ	3σ	
Within 100 pcm	2.52	2.52	2.60	26.09
Within 50 pcm	2.52	2.97	4.84	101.33

Table 6: Overall time-to-solution speedup of TRRM over traditional methods

	1σ	2σ	3σ
Within 100 pcm	10.4	10.4	10.0
Within 50 pcm	40.2	34.1	20.9

4.6. Assembly Testing

TRRM has been shown to have numerous advantages over traditional methods in terms of performance, memory efficiency, and reduced number of intersections required for convergence in the context of the IFBA pincell problem. However, these relative advantages may be subject to change when running on geometrically larger problems such as a reactor assembly scale problems. In this subsection, we will perform some of the same tests as done on the pincell IFBA problem with an assembly sized problem. The assembly is composed of a 17 x 17 lattice of pincells, with each pincell being the standard CASL VERA Problem 1E pincells used thus far and described in subsection 3.2. The outer borders of the assembly are reflective. This assembly is similar to the IFBA test assembly problem defined by Walker[26]. Using the same flat source region discretization for each pincell as has been used thus far in this paper, the assembly contains a total of 3,757 flat source regions.

While a lattice composed entirely of IFBA pincells is unlikely to be deployed in a real reactor, it can still serve as a good test case for IFBA geometry. The assembly scale IFBA problem allows for a realistic amount of "cancellation of error" to occur – wherein the flux and eigenvalue in a single IFBA pin may be mispredicted, but the overall eigenvalue across the full assembly will be much closer to the true answer as the errors in each pincell will be distributed around the mean. Thus, an assembly scale IFBA problem may not require as fine of a quadrature as compared to a single pincell (regardless of if a stochastic or fixed quadrature is used), as described by Walker[26]. To examine these affects relative to both TRRM and traditional methods, the assembly problem was constructed in OpenMOC and in our reference TRRM implementation for evaluation.

While performance characteristics (in terms of time per intersection) were unchanged when moving to the assembly problem, the memory efficiency of TRRM as compared to the traditional deterministic method was altered. The comparison given in Table 7 represents the memory requirements for the two

different methods in the context of the IFBA assembly problem. In the case of OpenMOC, these results are for 128 azimuthal angles and a track spacing of 0.01 cm, which was the coarsest quadrature configuration that could achieve less than 50 pcm eigenvalue error. This table shows that TRRM uses 75.9x less memory than traditional deterministic methods. The change in relative memory usage as compared to the pincell problem can be attributed to the change in surface area vs. volume for the problem, as a large contributor to memory usage for the traditional deterministic methods is in storage of the boundary angular flux data. As the domain becomes larger, there becomes less boundary angular flux to store relative to the amount of tracking and segment data.

Table 7: Memory usage comparison of TRRM and OpenMOC for a 2D IFBA assembly problem

	TRRM	OpenMOC
Boundary Angular Flux [MB]	-	170.53
Track and Segment Data [MB]	-	936.72
CSG & Nearest Neighbor Lists [MB]	1.68	-
Scalar Flux [MB]	12.84	7.34
XS Data [MB]	0.17	0.17
Total [MB]	14.69	1,114.76

The convergence properties of the traditional deterministic methods for the IFBA assembly problem are shown in Figures 17 and 18. For the larger assembly problem, the scalar flux was desired to converge to within 1e-4 RMS of the scalar flux, as shown in Figure 17. However, this produced a significant bias in the asymptotic results, so the convergence criteria for the deterministic method was increased to 1e-5 RMS on the scalar flux, as shown in Figure 18. These figures also display the accuracy of the volume correction method, which results in an overall significantly lower accuracy on the IFBA assembly problem.

Convergence and accuracy are also measured for TRRM and compared to the deterministic method, as shown in Figure 19. This plot shows both TRRM and OpenMOC converged to the desired 1e-4 on their scalar fluxes. The confidence intervals for TRRM were determined by running the code 100 times independently for each configuration and tracking the mean and

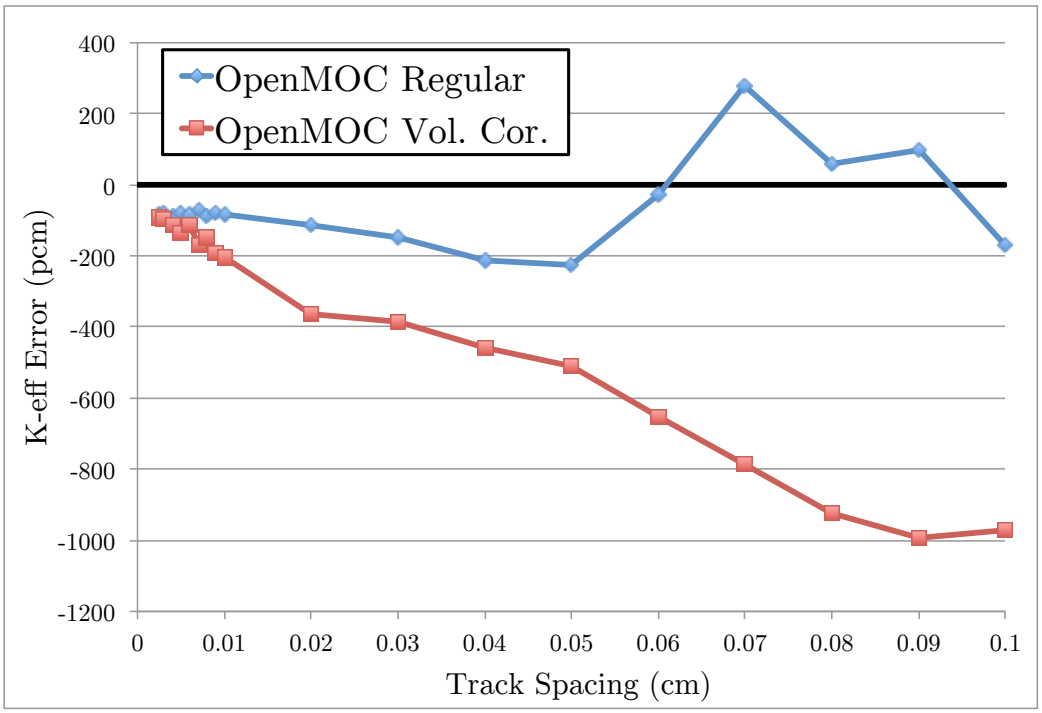


Figure 17: Traditional deterministic method accuracy vs. track spacing on the IFBA assembly problem. Results were converged to 1e-4 RMS accuracy on the scalar flux.

standard deviation of the output. As can be seen from this plot, the increase in problem size has increased the accuracy of both methods, with TRRM showing an even greater algorithmic efficiency gain. The coarsest TRRM configuration uses 400 rays at 100 cm distance with 10 cm dead zone length, producing 2.27e+10 intersections with a standard deviation of 9.44 pcm (resulting in a 3σ confidence of $\pm$ 28.3 pcm). Convergence to a smaller RMS of the scalar flux was not required for TRRM, as the eigenvalue had already highly converged to a small standard deviation by the time 1e-4 RMS of the scalar flux was achieved. An additional plot showing OpenMOC converged to 1e-5 RMS and TRRM converged to 1e-4 RMS is shown in Figure 20, which greatly improved the accuracy of OpenMOC.

There are a number of key data points to highlight in Figure 20. For the traditional deterministic method to achieve less than 50 pcm error, a track spacing of 0.01 cm was required resulting in 4.0e+11 intersections. To achieve

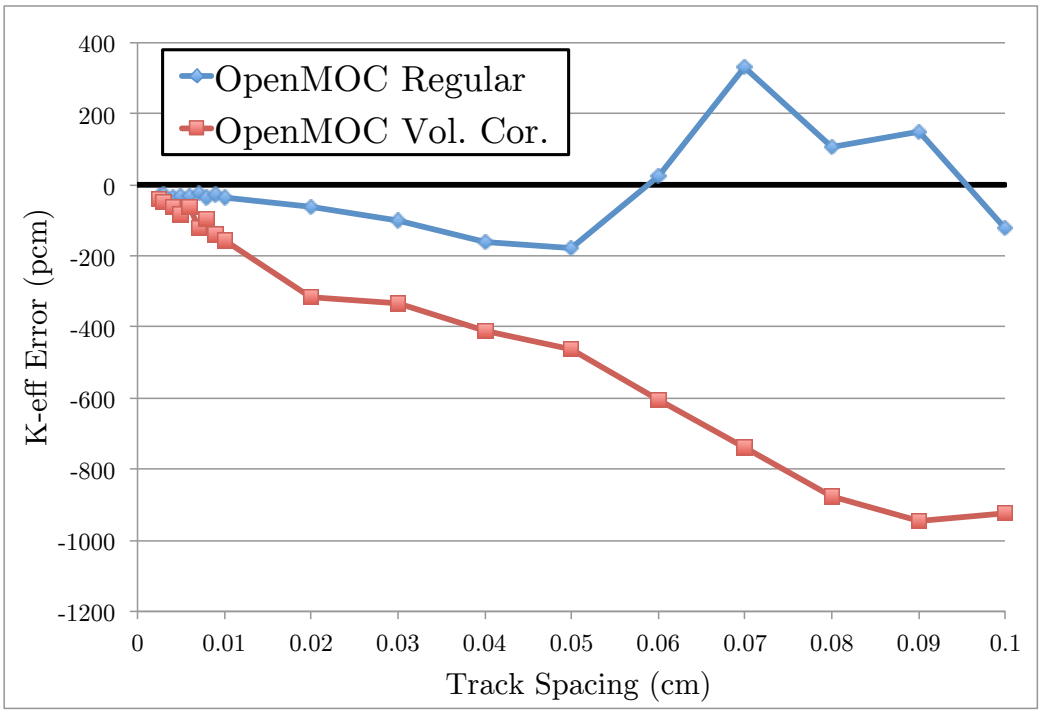


Figure 18: Traditional deterministic method accuracy vs. track spacing on the IFBA assembly problem. Results were converged to 1e-5 RMS accuracy on the scalar flux. Note the reduction in asymptotic solution bias as compared to Figure 17.

less than 50 pcm error, with a confidence level of 5σ, TRRM only required 2.27e+10 intersections. This means that TRRM was able to offer equivalent accuracy to OpenMOC with 5σ confidence while reducing the number of intersections required by a factor of 17.6x.

5. 3D Testing

In the previous section the performance, accuracy, and convergence characteristics of The Random Ray Method were studied. For the purposes of comparing TRRM to traditional algorithms, all studies were performed using a 2D problem wherein our native 3D implementation was altered so as to always sample the same polar angle. This allowed us to easily compare the accuracy of the code to traditional 2D methods using the same single polar angle. In this section, we will present fully 3D results using TRRM for a

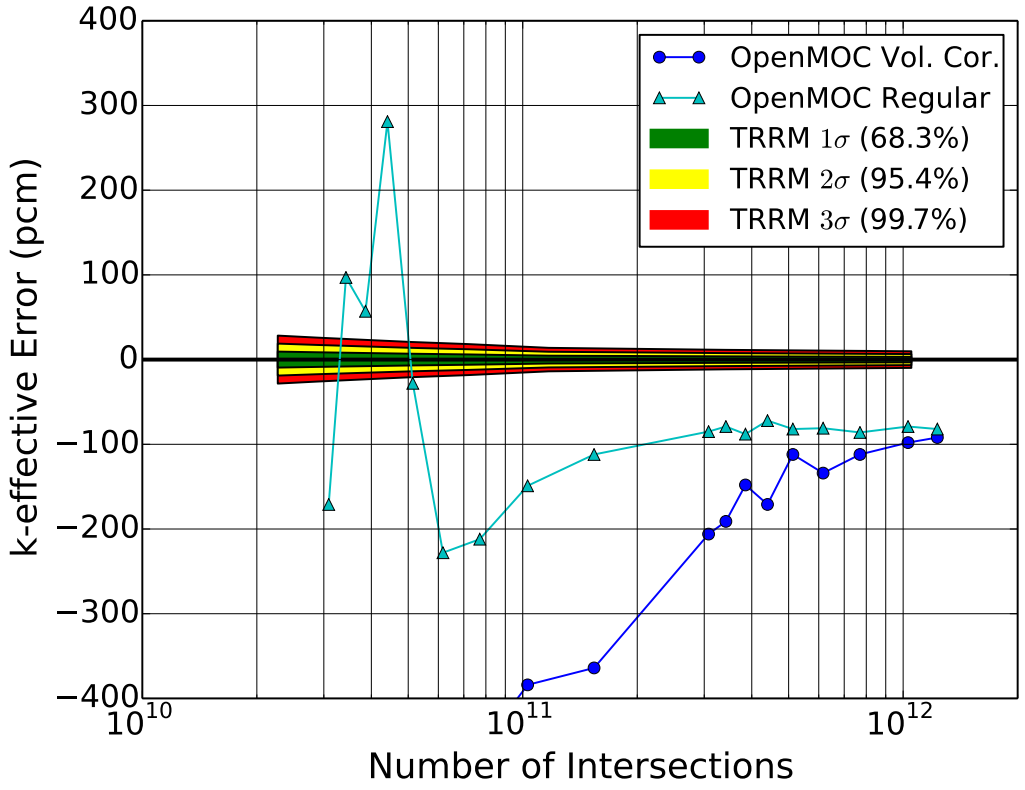


Figure 19: Comparison of TRRM to traditional methods for a 2D IFBA assembly problem. This plot shows eigenvalue error versus total number of intersections required for convergence. Results were converged to 1e-4 RMS accuracy on the scalar flux for both methods.

basic 3D pin cell geometry in order to ensure that the method is capable of working in 3D.

Our test case for this problem was a simplified pin cell without IFBA, having only 2 regions: fuel and moderator. The gap and clad materials have been smeared together with the fuel pellet to generate a single "fuel" region. Cross sections were again generated using OpenMC. The standard pin cell is 300 cm in height, with a 10 cm water reflector on both the top and bottom. The vertical length of the fueled pin cell was subdivided into 100 axial zones. The radial boundaries are all reflective while the very top and bottom (bordering the water reflectors) have a vacuum boundary condition. The axial fast and thermal fluxes (delineated at 1.0 eV) are plotted in Figure 21, which

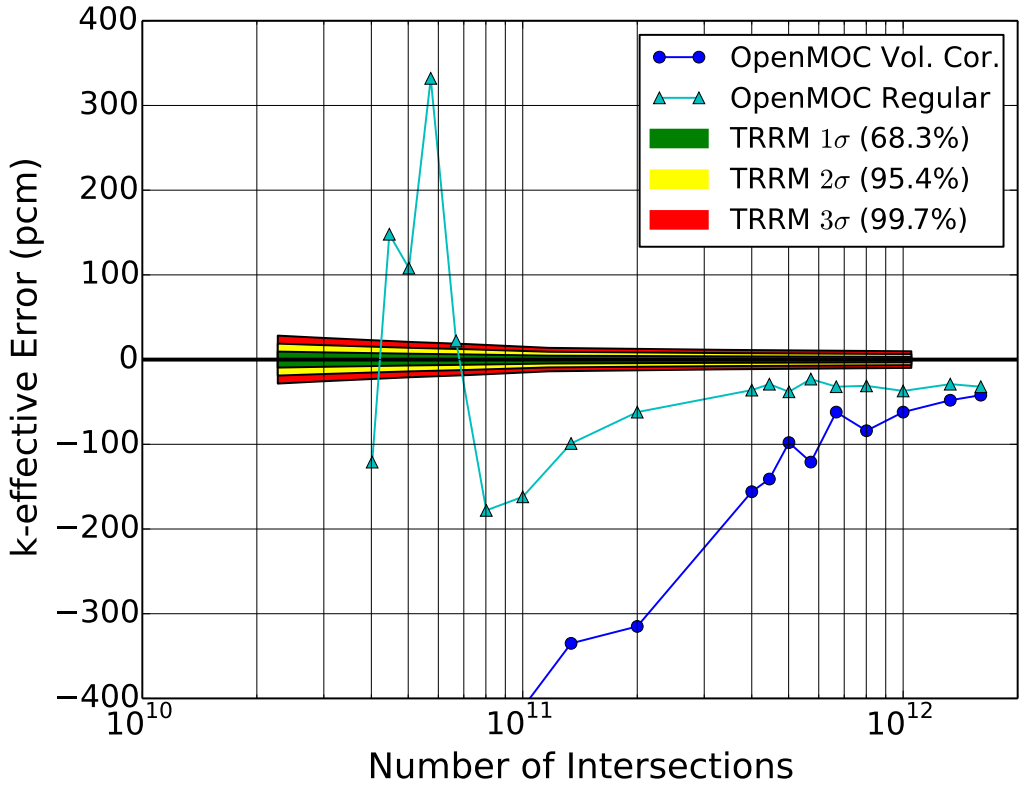


Figure 20: Comparison of TRRM to traditional methods for a 2D IFBA assembly problem. This plot shows eigenvalue error versus total number of intersections required for convergence. Results were converged to 1e-4 RMS accuracy on the scalar flux for TRRM and 1e-5 RMS of the scalar flux for OpenMOC. Note the reduction in asymptotic solution error in the OpenMOC case as compared to Figure 19.

appear as expected for this type of problem.

Performance characteristics of the algorithm were identical to those reported in subsection 4.2. This is due to the ray tracer always operating in 3D space, even when simulating a 2D problem, as the reduction in dimensionality is achieved only by changing the way rays are sampled. In 2D, polar angles are sampled from the single cosine of the angle TY quadrature (i.e., 0.602399), rather than freely in space (i.e., random cosine of the angle between -1 and 1). The implication of this is that no additional work or overhead is required for 3D simulation in our reference TRRM implementation. These results show

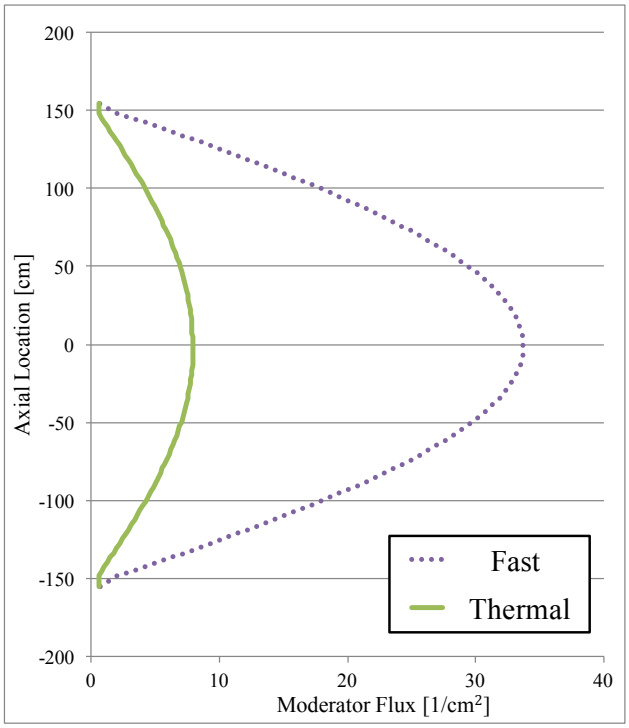


Figure 21: Axial flux profile in the moderator flat source region of the finite pin cell.

that TRRM is capable of successfully simulating 3D problems without having to add any additional complexity to Algorithms 1, 2, and 3.

6. Conclusions

The Method of Characteristics (MOC) is an accurate method that is used in a number of fields of computational physics to numerically estimate the solution of a PDE by reducing it to a family of ODEs that are defined along characteristic tracks through the domain of the problem space. This paper has introduced a new method based on MOC, known as The Random Ray Method (TRRM), that uses a stochastic rather than deterministic process to select which characteristics to solve along. The new method may be applicable to a range of fields of computational physics that currently utilize MOC techniques, such as neutron transport, gamma ray transport, and astrophysics radiative transfer. In order to provide a preliminary assessment

of the general utility of TRRM, the new method was implemented in the context of an example neutron transport application and tested on a set of reactor physics benchmark problems. In the context of neutron transport, this paper has demonstrated the following general advantages of TRRM compared to traditional deterministic MOC methods:

- Geometrically flexible, does not require any level of axial symmetry. Works with any constructive solid geometry (CSG) representation of a reactor.
- Does not require angular flux to be stored between power iterations, or explicit storage of a tracking file, greatly reducing memory requirements.
- Scales up easily to large 3D problems (such as full size reactors) due to low memory overhead
- Simple to implement, no additional code complexity required for efficient 3D scaling

In this study, we have also performed a preliminary assessment of the practical performance merits of the TRRM algorithm in comparison to traditional deterministic MOC methods as implemented in a reference deterministic MOC application. This performance analysis was done to ensure the performance characteristics of the new algorithm on a real benchmark problem are well understood and that ray tracing costs are not prohibitive. In the context of the CASL VERA Core Physics Benchmark Problem number 1E[25] (a 2D IFBA pincell problem), we were able to find that TRRM:

- Outperforms traditional methods in a "time per intersection" test, running 2.3x (serial) to 14.2x (in parallel) faster than traditional methods on our test system.
- Runs at 2.69 ns to 4.84 ns per intersection (in serial), depending on computer architecture. This is very fast compared to the standard criteria of 10 ns per intersection for highly optimized production 2D MOC solvers.
- Uses 652x less memory than traditional methods, allowing for considerably higher cache efficiency, speed, and scalability.

- Converges more quickly than deterministic methods, reducing the total number of intersections required by a factor of 1.6x to 18.6x depending on desired accuracy and confidence levels.
- Offers a shorter runtime than traditional methods, with a time-to-solution that is 10x to 40x faster depending on desired accuracy and confidence levels.

Additionally, in the context of a 2D IFBA assembly problem, we were able to find that TRRM also:

- Achieves similar performance characteristics (such as time per intersection) as the pincell problem.
- Uses 75.9x less memory that traditional deterministic methods.
- Converges more quickly than deterministic methods, reducing the total number of intersections required by a factor of 17.6x.

While this analysis is limited in scope to a single set of reference implementations and a limited field of test problems in reactor physics, the preliminary performance and convergence characteristics of TRRM are compelling given the method's geometrically flexible nature and low memory requirements. Overall, the benefits of TRRM are significant enough to potentially allow for high fidelity simulations of large problems that were previously impractical when using traditional deterministic MOC methods. In this light, TRRM shows great promise at offering improvements in a variety of transport simulation fields such as neutron transport (as demonstrated in this paper), gamma ray transport, and radiative transfer.

Acknowledgement

Special thanks to the entire Computational Reactor Physics Group at MIT for their help with this work. Particular thanks to Derek Gaston for acting as a sounding board while working on developing TRRM and this paper. Also special thanks to William Boyd, Samuel Shaner, and Geoff Gunow for their help on this project. Development of this method would not have been possible without the OpenMOC code. It proved an invaluable tool to learn about the traditional MOC algorithm and acted as an ideal basis for comparison.

We gratefully acknowledge the computing resources provided and operated by the Joint Laboratory for System Evaluation (JLSE) at Argonne National Laboratory.

This work was supported by the Office of Advanced Scientific Computing Research, Office of Science, U.S. Department of Energy, under Contract DE-AC02-06CH11357. The submitted manuscript has been created by the University of Chicago as Operator of Argonne National Laboratory ("Argonne") under Contract DE-AC02-06CH11357 with the U.S. Department of Energy. The U.S. Government retains for itself, and others acting on its behalf, a paid-up, nonexclusive, irrevocable worldwide license in said article to reproduce, prepare derivative works, distribute copies to the public, and perform publicly and display publicly, by or on behalf of the Government.

References

[1] R. Sanchez, Prospects in deterministic three-dimensional whole-core transport calculations, Nuclear Engineering and Technology 44 (5) (2012) 113–150. doi:10.5516/NET.01.2012.501.

[2] W. Boyd, S. Shaner, L. Li, B. Forget, K. Smith, The OpenMOC method of characteristics neutral particle transport code, Annals of Nuclear Energy 68 (2014) 43–52. doi:10.1016/j.anucene.2013.12.012.
URL http://dx.doi.org/10.1016/j.anucene.2013.12.012

[3] R. M. Ferrer, J. Rhodes, Extension of linear source MOC methodology to anisotropic scattering in CASMO5, in: PHYSOR 2014 - The Role of Reactor Physics toward a Sustainable Future, 2014.

[4] M. Eklund, M. Alamaniotis, H. Hernandez, T. Jevremovic, Method of characteristics - a review with applications to science and nuclear engineering computation, Progress in Nuclear Energy 85 (2015) 548–567. doi:10.1016/j.pnucene.2015.05.002.

[5] K. S. Kim, S. G. Hong, Gamma transport and diffusion calculation capability coupled with neutron transport simulation in KARMA 1.2, Annals of Nuclear Energy 57 (2013) 59 – 67. doi:http://dx.doi.org/10.1016/j.anucene.2013.01.047.

[6] P. Humbert, B. Méchitoua, Gamma ray transport simulations using SGaRD code, EPJ Nuclear Sci. Technol. 3 (2017) 9. doi:10.1051/epjn/2017006.

[7] T. Simeonov, HELIOS2: Benchmarking against experiments for hexagonal and square lattices, in: 19th AER Symposium on VVER Reactor Physics and Reactor Safety, 2009.
URL https://inis.iaea.org/search/search.aspx?orig_q=RN:40108133

[8] A. Rosen, M. Krumholz, J. Oishi, A. Lee, R. Klein, Hybrid adaptive ray-moment method (HARM2): A highly parallel method for radiation hydrodynamics on adaptive grids, Journal of Computational Physics 330 (2017) 924 – 942. doi:http://dx.doi.org/10.1016/j.jcp.2016.10.048.

[9] L. Buntemeyer, R. Banerjee, T. Peters, M. Klassen, R. E. Pudritz, Radiation hydrodynamics using characteristics on adaptive decomposed domains for massively parallel star formation simulations, New Astronomy 43 (2016) 49 – 69. doi:http://dx.doi.org/10.1016/j.newast.2015.07.002.

[10] E. Rijkhorst, T. Plewa, A. Dubey, G. Mellema, Hybrid characteristics : 3D radiative transfer for parallel adaptive mesh refinement hydrodynamics, Astronomy and Astrophysics 452 (2006) 907–920.

[11] J. H. Wise, T. Abel, enzo+moray: radiation hydrodynamics adaptive mesh refinement simulations with adaptive ray tracing, Monthly Notices of the Royal Astronomical Society 414 (4) (2011) 3458. doi:http://dx.doi.org/10.1111/j.1365-2966.2011.18646.x.

[12] J. R. Tramm, G. Gunow, T. He, K. S. Smith, B. Forget, A. R. Siegel, A task-based parallelism and vectorized approach to 3D Method of Characteristics (MOC) reactor simulation for high performance computing architectures, Computer Physics Communications 202 (2016) 141 – 150. doi:http://dx.doi.org/10.1016/j.cpc.2016.01.007.

[13] G. Gunow, J. R. Tramm, B. Forget, K. Smith, SimpleMOC – A performance abstraction for 3D MOC, in: ANS&MC 2015 – Joint International Conference on Mathematics and Computation (M&C), Supercomputing

in Nuclear Applications (SNA) and the Monte Carlo (MC) Method, Nashville, 2015.

[14] J. M. Bull, Single node performance analysis of applications on HPCx, Tech. Rep. 0703, HPCx Edinburgh Parallel Computing Centre, University of Edinburgh (2007).
URL http://www.hpcx.ac.uk/research/hpc/technical_reports/HPCxTR0703.pdf

[15] J. R. Tramm, A. R. Siegel, Memory bottlenecks and memory contention in multi-core Monte Carlo transport codes, Annals of Nuclear Energy 82 (2015) 195 – 202. doi:http://dx.doi.org/10.1016/j.anucene.2014.08.038.

[16] B. Kochunas, A hybrid parallel algorithm for the 3-D Method of Characteristics solution of the Boltzmann Transport Equation on high performance computing clusters, Ph.D. Thesis, University of Michigan, Department of Nuclear Engineering and Radiological Sciences (2013).

[17] G. Palmiotti, M. Smith, C. Rabiti, M. Leclere, D. Kaushik, A. Siegel, B. Smith, E. Lewis, UNÌC: Ultimate neutronic investigation code, in: M&C + SNA 2007 – Joint International Topical Meeting on Mathematics & Computation and Supercomputing in Nuclear Applications, Monterey, California, 2007.

[18] M. A. Smith, D. Kaushik, A. Wollaber, W. S. Yang, B. Smith, C. Rabiti, G. Palmiotti, Recent research progress on UNÌC at Argonne National Laboratory, in: International Conference on Mathematics, Computational Methods & Reactor Physics, Saratoga Springs, New York, 2009.

[19] H. Prabha, G. Marleau, A. Hébert, Tracking algorithms for multi-hexagonal assemblies (2D and 3D), Annals of Nuclear Energy 69 (2014) 175–182. doi:10.1016/j.anucene.2014.01.018.
URL http://linkinghub.elsevier.com/retrieve/pii/S0306454914000267

[20] H. Prabha, G. Marleau, Computation of 3D neutron fluxes in one pin hexagonal cell, Annals of Nuclear Energy 56 (2013) 1–6. doi:10.1016/j.anucene.2012.12.023.
URL http://dx.doi.org/10.1016/j.anucene.2012.12.023

[21] B. W. Kelley and E. W. Larsen, 2D/1D approximations to the 3D neutron transport equation. I: Theory, in: International Conference on Mathematics and Computational Methods Applied to Nuclear Science and Engineering, Knoxville, TN, USA, 2013.

[22] D. Sciannandrone, S. Santandrea, R. Sanchez, Optimized tracking strategies for step MOC calculations in extruded 3D axial geometries, Annals of Nuclear Energy 87, Part 1 (2016) 49 – 60, special Issue of The 3rd International Conference on Physics and Technology of Reactors and Application. doi:http://dx.doi.org/10.1016/j.anucene.2015.05.014.
URL http://www.sciencedirect.com/science/article/pii/S0306454915002728

[23] W. Boyd, Massively parallel algorithms for Method of Characteristics neutral particle transport on shared memory computer architectures, M.S. Thesis, Massachusetts Institute of Technology, Department of Nuclear Science and Engineering (2014).
URL http://hdl.handle.net/1721.1/87494

[24] R. M. Ferrer, J. Rhodes, K. Smith, Linear source approximation in CASMO5, in: PHYSOR 2012 – Advances in Reactor Physics – Linking Research, Industry, and Education, 2012.

[25] VERA core physics benchmark progression problem specifications, revision 4, Tech. Rep. CASL-U-2012-0131-004, Consortium for Advanced Simulation of Light Water Reactors (CASL) (2014).

[26] E. D. Walker, Modeling integral fuel burnable absorbers using the method of characteristics. Master's Thesis, University of Tennessee, 2014.
URL http://trace.tennessee.edu/utk_gradthes/3191

[27] P. K. Romano, B. Forget, The OpenMC Monte Carlo particle transport code, Annals of Nuclear Energy 51 (C) (2013) 274–281.

[28] B. Herman, Monte Carlo and thermal hydraulic coupling using low-order nonlinear diffusion acceleration, PhD Thesis, Massachusetts Institute of Technology, Department of Nuclear Science and Engineering (2014).
URL http://hdl.handle.net/1721.1/95525

[29] A. Yamamoto, M. Tabuchi, N. Sugimura, T. Ushio, M. Mori, Derivation of optimum polar angle quadrature set for the Method of Characteristics

based on approximation error for the Bickley Function, Journal of Nuclear Science and Technology 44 (2) (2007) 129–136. doi:http://dx.doi.org/10.1080/18811248.2007.9711266.

[30] R. Aggarwal, A. Caldwell, Error bars for distributions of numbers of events, The European Physical Journal Plus 127 (2) (2012) 1–8. doi:http://dx.doi.org/10.1140/epjp/i2012-12024-0.